

目录

TermSync: 终端协同代码编辑器	4
1 项目概述	4
2 项目背景	6
3 功能描述	7
3.1 多人连接与文档同步	7
3.2 字符级编辑操作	7
3.3 ACK 驱动的客户发送队列	8
3.4 服务端主导 OT 与全局版本	8
3.5 保存、自动保存与编辑日志	9
3.6 历史裁剪与重同步	10
4 设计思路	10
4.1 总体架构	10
4.2 网络协议设计	10
4.3 文档锁与全局顺序	11
4.4 OT 位置变换规则	11
4.5 客户端状态机	12
4.6 自动保存进程	12
5 测试与验证	12
5.1 单元测试	12
5.2 本地验证	13
5.3 部署验证	16
6 问题与解决方案	19
6.1 并发操作顺序不确定	19
6.2 旧版本操作位置失效	19
6.3 连续输入容易丢失或乱序	19
6.4 慢客户端可能阻塞服务端	20

6.5	历史无限增长	20
6.6	文件保存可能覆盖旧内容	20
6.7	中文字符显示宽度	20
7	项目不足与改进方向	20
8	代码展示	21
8.1	协议与基础数据结构	21
8.2	服务端权威文档与 OT	26
8.3	服务端连接、广播与保存触发	32
8.4	客户端状态机与终端界面	41
8.5	自动保存、备份与日志	51
8.6	单元测试	54
Linux 平台下的基本命令		62
1	文件和目录操作	62
2	文件内容查看与处理命令	63
3	系统信息查看命令	65
4	权限相关命令	68
5	管道符组合命令	68
6	实验小结	70
Linux 下的 Shell 编程案例		71
1	程序功能说明	71
2	Shell 脚本执行方法	71
3	程序实现思路	72
4	案例展示	72
5	Shell 程序调试方法	76
6	实验小结	79
7	源代码与题库展示	79
7.1	Shell 脚本	79

7.2	题库文件	84
-----	----------------	----

TermSync: 终端协同代码编辑器

1 项目概述

本项目名为 TermSync，是一个使用 Python 标准库实现的终端多人协作代码编辑器。项目的目标不是复刻完整的图形化 IDE，而是在 Linux 环境下实现一个可实际运行、可多人连接、可同步编辑、可保存和可测试验证的轻量级协作编辑系统。用户在终端中运行客户端，通过 TCP 连接到服务端；服务端维护唯一权威文档状态，并负责对所有编辑操作进行排序、变换、应用和广播。

我刻意减少第三方依赖，主要使用 Python 标准库完成系统实现。网络通信基于 `socket`，并发处理使用 `threading` 和 `multiprocessing`，终端界面使用 `curses`，消息格式和持久化则基于 `json`、`pathlib` 等标准库模块。项目没有依赖现成的 Web 框架、协同编辑库或数据库系统，因此网络协议、客户端状态机、服务端 OT 变换、广播同步和保存机制都由项目代码自行实现，更能体现 Linux 程序设计中进程、线程、文件和网络等基础能力的直接使用。

本项目覆盖了 Linux 程序设计中的多个核心主题：网络编程、线程创建与同步、进程间通信、文件系统操作、终端界面编程和自动化测试。服务端使用 `socket` 监听客户端连接，为每个客户端创建独立线程；文档对象使用锁保护共享状态；自动保存和日志写入被拆分到独立进程中，通过 `multiprocessing.Queue` 与主服务端通信；客户端使用 `curses` 构建终端交互界面。

项目采用“服务端主导 OT (Operational Transformation)”的同步方案。客户端只负责将键盘输入转换为字符级编辑操作，并按 ACK 节奏发送到服务端。服务端作为唯一权威状态源，在临界区内为每个操作分配递增的 `server_version`，并根据历史操作对旧版本操作进行位置变换。这样既降低了客户端实现复杂度，也能清晰展示并发编辑时的一致性处理过程。

在实际部署中，服务端运行在我的个人 Linux 服务器上。这台服务器是拥有独立公网 IP 的阿里云 ECS 服务器，并已经绑定域名 `glance02.xyz`，因此客户端后续不需要直接记忆服务器公网 IP，而是通过域名连接服务端。例如服务端在 Linux 服务器上监听 `0.0.0.0:8765`，客户端可以使用 `glance02.xyz:8765` 加入同一个协作文档。由于我的两台电脑都是 Windows 系统，客户端实际运行在 Windows 的 WSL (Windows Subsystem for Linux) 环境中。WSL 提供了较完整的 Linux 用户空间环境，支持 Python、`curses` 和 TCP 网络编程，因此客户端可以直接在 WSL 终端中运行，并通过公网域名

连接到服务器。

```
root@iZbp1la51jwdv4pjbv2r1oZ ~ /Linux_Class (code)# python3 server.py --host 0.0.0.0 --port 8765 --file shared.py
Collaborative server listening on 0.0.0.0:8765
Document: shared.py
JOIN client=glance address=('36.156.65.4', 2279)
JOIN client=sputnik address=('182.96.176.182', 3833)
OP v1 client=sputnik insert pos=70 char='\n' base=0
OP v2 client=sputnik insert pos=71 char='I' base=1
OP v3 client=sputnik insert pos=72 char=' ' base=2
OP v4 client=sputnik insert pos=73 char='a' base=3
OP v5 client=sputnik insert pos=74 char='m' base=4
OP v6 client=sputnik insert pos=75 char=' ' base=5
OP v7 client=sputnik insert pos=76 char='S' base=6
OP v8 client=sputnik insert pos=77 char='p' base=7
OP v9 client=sputnik insert pos=78 char='u' base=8
OP v10 client=sputnik insert pos=79 char='t' base=9
OP v11 client=sputnik insert pos=80 char='n' base=10
OP v12 client=sputnik insert pos=81 char='i' base=11
OP v13 client=sputnik insert pos=82 char='k' base=12
OP v14 client=glance insert pos=83 char=',' base=13
OP v15 client=glance insert pos=84 char=' ' base=14
OP v16 client=glance insert pos=85 char='a' base=15
OP v17 client=glance insert pos=86 char='n' base=16
OP v18 client=glance insert pos=87 char='d' base=17
OP v19 client=glance insert pos=88 char=' ' base=18
OP v20 client=glance insert pos=89 char='i' base=19
OP v21 client=glance delete pos=89 base=20
```

图 1 服务端运行界面

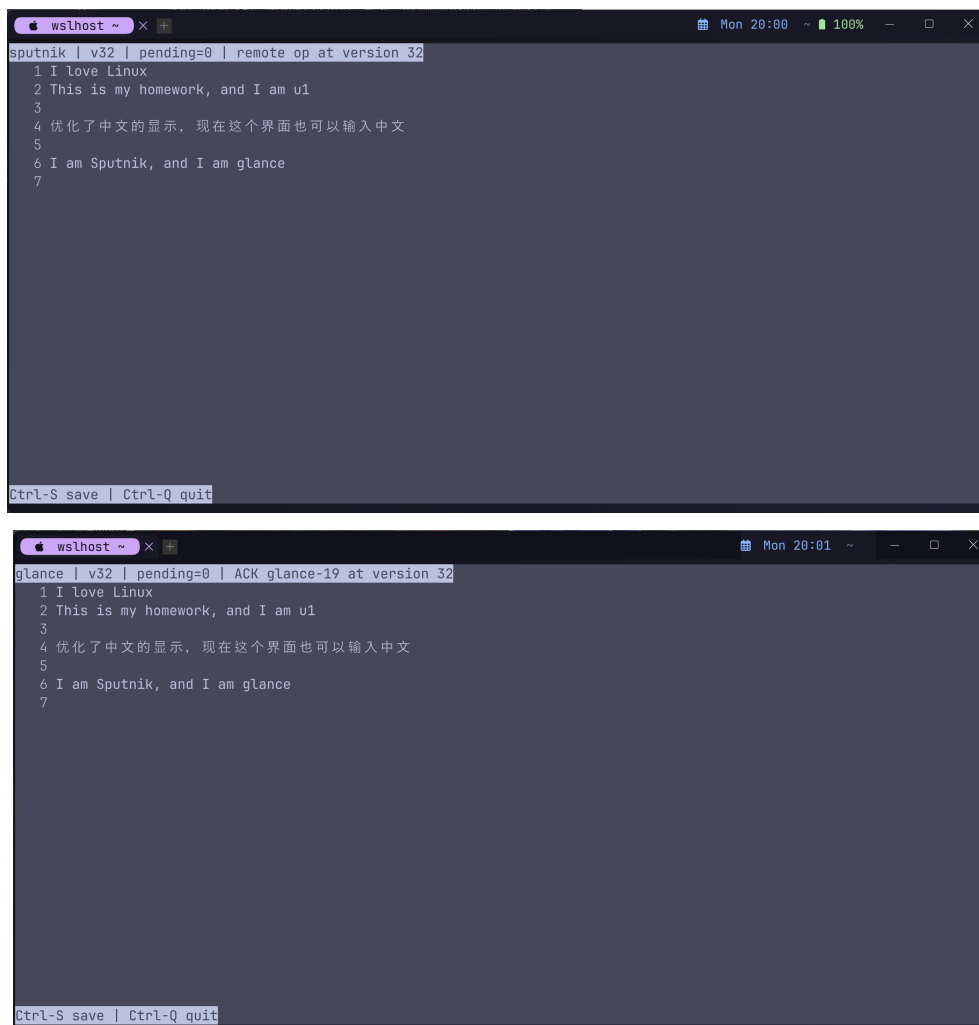


图 2 两个客户端连接服务端

前面三张运行截图展示了 TermSync 的基本启动和连接流程。第一张图中，服务端

已经在 Linux 服务器上启动并监听指定端口，终端输出中可以看到服务端正在等待客户端接入。后两张图分别展示两个不同客户端通过域名连接到同一个服务端实例，并进入终端编辑界面；这说明服务端可以同时维护多个客户端连接，新加入的客户端也能够获得当前共享文档状态，为后续的同步编辑、广播更新和保存验证提供了运行基础。

2 项目背景

多人协作编辑是一个典型的分布式一致性问题。多个用户可能在同一时间对同一份文本进行修改，如果系统只按照客户端本地光标位置直接修改文档，很容易出现字符位置错乱、删除目标错误、不同客户端内容不一致等问题。例如初始文本为 `hello world`，一个用户在位置 5 连续插入多个字符，另一个用户仍然基于旧版本请求删除位置 5 的字符。如果服务端不做额外处理，第二个用户想删除的字符和服务端当前文档中的位置就不再对应。

传统协作编辑系统常见的解决方案包括 OT 和 CRDT。本项目选择 OT：每个编辑操作携带自己基于的文档版本，服务端根据该版本之后已经发生的历史操作修正位置，最后将操作应用到当前文档。与更复杂的全客户端协同 OT 不同，本项目采用服务端主导方案，将全局排序和位置变换集中放在服务端，从而使实现更清晰，也更容易验证正确性。

本项目也具有明确的 Linux 课程背景。首先，服务端是一个典型的 TCP 长连接服务，涉及 socket 创建、地址绑定、监听、接受连接和消息收发。其次，每个客户端连接由独立线程处理，多个线程会同时访问共享文档，因此必须使用锁保护临界区。再次，自动保存和日志写入使用独立进程完成，主进程通过队列向工作进程发送快照、保存和日志事件，体现了进程间通信的使用。最后，文件加载、原子写入、备份文件、自动保存文件和编辑日志都体现了 Linux 文件系统编程的基本实践。

此外，终端编辑器本身也与 Linux 使用场景贴合。客户端基于 `curses` 运行在终端中，支持方向键移动、普通字符输入、回车换行、Backspace 删除、`Ctrl-S` 保存和 `Ctrl-Q` 退出。相比网页前端，这种实现更接近 Linux 命令行环境下的工具开发，也便于在服务器和 WSL 中演示。

从项目边界上看，TermSync 更偏向教学型协作编辑器。它重点展示字符级同步、网络通信、并发控制、文件持久化和自动化测试，而不追求语法高亮、代码补全、复杂撤销栈等完整 IDE 功能。这样的取舍使项目能够把主要篇幅集中在 Linux 程序设计课

程强调的系统调用、进程线程、终端交互和文件操作上。

3 功能描述

3.1 多人连接与文档同步

服务端通过 TCP socket 对外提供协作编辑服务。客户端连接后首先发送 JOIN 消息，服务端为其分配或确认 `client_id`，随后发送 WELCOME 和完整的 DOC_STATE。这样新加入的客户端可以立即得到当前文档内容、版本号和历史起始版本。

每个客户端连接由服务端中的一个独立线程处理。线程持续读取该客户端发来的 JSON Lines 消息，并根据消息类型分发到不同处理逻辑。编辑操作使用 OP 消息提交，保存请求使用 SAVE 消息提交，客户端也可以主动发送 REQUEST_DOC 请求完整文档。

3.2 字符级编辑操作

项目支持的核心编辑操作是字符级 `insert` 和 `delete`。插入操作包含插入位置和单个字符，删除操作包含删除位置。换行符 `\n` 被视为普通字符处理，因此多行文本可以自然表示为一个字符串。

客户端终端界面将用户按键转换为这些基础操作。普通可显示字符会生成 `insert`；回车会生成插入换行符；Backspace 会生成删除光标前一个字符的 `delete`。客户端本地光标会随着输入移动，但文档内容最终以服务端返回的 ACK 或远程广播为准。

这些操作在代码中的定义集中在代码仓库根目录下的 `document.py: VALID_KINDS` 限定操作类型只能是 `insert`、`delete` 和 `noop`，`Operation` 数据类保存 `kind`、`pos`、`char` 和可选的 `reason`。客户端在 `client.py` 的按键处理函数中把普通字符、回车和 Backspace 转换成 `Operation`，再由 `client_state.py` 封装成网络层的 OP 消息。服务端终端上看到的 OP v... 并不是另一种新的操作模型，而是 `server.py` 中日志格式化函数对服务端事件的可读化输出。

例如一条服务端输出 `OP v7 client=sputnik insert pos=76 char='S' base=6` 表示：服务端把这次操作确认为全局第 7 个版本，操作来自 `sputnik` 客户端，最终权威操作是在位置 76 插入字符 `S`，而客户端提交时基于的是版本 6。换行也按普通字符参与协同编辑，所以回车会显示为 `char='\n'`。

字符可以输入英文、数字、符号，也可以输入中文等宽字符。客户端使用 Python 的 `unicodedata` 模块，通过 `east_asian_width` 判断字符宽度，并提供显示列和字符

列之间的转换函数，确保光标移动和文本显示在终端中正确对齐。

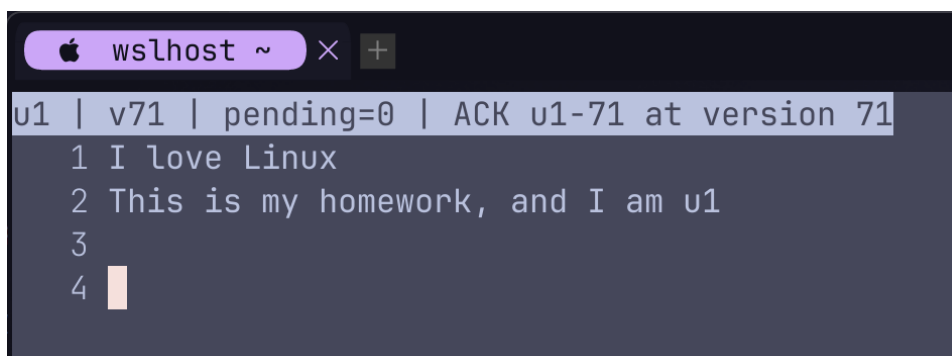
A terminal window titled 'wslhost ~' with a dark background. The status bar at the top shows 'u1 | v71 | pending=0 | ACK u1-71 at version 71'. Below the status bar, there is a list of input lines: '1 I love Linux', '2 This is my homework, and I am u1', '3', and '4' followed by a cursor. The text is left-aligned and uses a monospaced font.

图 3 服务端英文输入

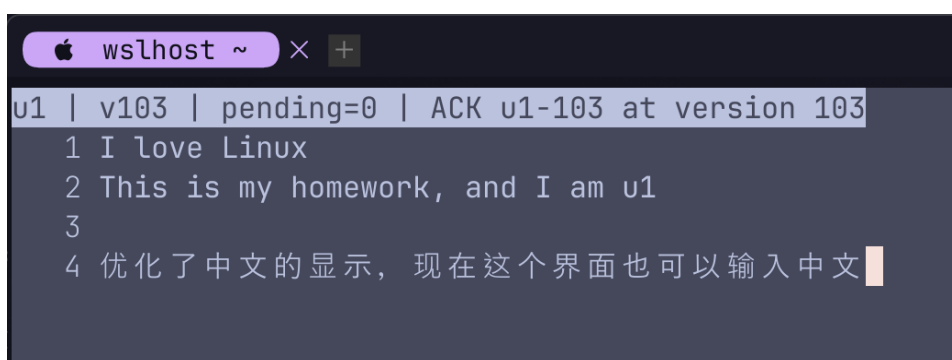
A terminal window titled 'wslhost ~' with a dark background. The status bar at the top shows 'u1 | v103 | pending=0 | ACK u1-103 at version 103'. Below the status bar, there is a list of input lines: '1 I love Linux', '2 This is my homework, and I am u1', '3', and '4 优化了中文的显示，现在这个界面也可以输入中文' followed by a cursor. The text is left-aligned and uses a monospaced font.

图 4 服务端中文输入

3.3 ACK 驱动的客户发送队列

为了避免用户连续快速输入时丢失按键，客户端维护 `pending_queue`。每次用户输入产生的操作都会先进入队列。与此同时，客户端同一时刻最多只允许存在一个 `inflight_op`，也就是已经发送但尚未收到服务端 ACK 的操作。

当当前操作收到 ACK 后，客户端使用 ACK 中的权威操作更新本地文档，再从队列中取出下一个操作发送。这个设计的好处是客户端不需要实现复杂的 OT 逻辑，只需要维护本地队列和版本号；所有并发冲突都交给服务端处理。

3.4 服务端主导 OT 与全局版本

服务端的文档对象维护当前文本、当前版本号、有限历史记录和文档锁。所有编辑操作只有进入文档锁保护的临界区后，才会被分配新的 `server_version`。这个版本号既表示文档版本，也表示操作在全局历史中的顺序。

当客户端提交的 `base_version` 落后于服务端当前版本时，服务端会从历史记录中

取出该版本之后的所有操作，逐条对客户端操作进行位置变换。例如历史中已经发生过一次插入，那么位于插入点之后的操作位置需要右移；如果两个删除操作删除同一个字符，后到达的删除会被转换为 `noop`。

3.5 保存、自动保存与编辑日志

服务端启动时会加载指定文档文件。如果文件不存在，则以空文档开始。成功处理编辑操作后，服务端会把最新快照发送给自动保存进程。自动保存进程定期写入 `.autosave` 文件，避免每次按键都直接触发磁盘写入。

用户在客户端按下 `Ctrl-S` 后，客户端发送 `SAVE` 消息。服务端收到后将当前快照投递给自动保存进程，由工作进程写入正式文件。为了降低误覆盖风险，正式保存前会生成 `.bak` 备份文件。同时，服务端会记录加入、离开、操作、自动保存和手动保存等事件，形成 JSON Lines 格式的编辑日志。



图 5 服务端编辑日志与自动保存记录

图 5 中展示了服务端运行目录中的保存产物和编辑日志。左侧可以看到正式文档 `shared.py`、自动保存文件 `shared.py.autosave`、备份文件 `shared.py.bak` 以及编辑日志 `shared.py.edit.log` 同时存在，说明服务端把文档内容、自动保存快照、覆盖前备份和事件记录分开管理。右侧打开的日志文件由多条 JSON 事件组成，每条记录包含 `event`、`path`、`timestamp` 和 `version` 等字段；其中 `autosave` 事件表示自动保存进程已经按固定间隔将当前快照写入 `.autosave` 文件。后续发生手动保存或编辑操作时，同一日志文件还会继续追加 `manual_save`、`operation` 等记录，用于追踪服务端文档状态的变化过程。

3.6 历史裁剪与重同步

为了避免服务端无限保存历史操作，文档对象使用有限长度的历史记录，默认保留最近 1000 条操作。服务端同时维护 `history_start_version`，表示当前历史仍然能够支持 OT 变换的最早基准版本。

如果某个客户端提交的 `base_version` 已经过旧，服务端无法可靠地根据有限历史进行变换，此时会返回 `RESYNC_REQUIRED`，并发送完整的 `DOC_STATE`。客户端收到后清空当前未确认操作，并请求或接受完整文档，从而恢复到服务端权威状态。

4 设计思路

4.1 总体架构

项目整体采用客户端-服务端架构。客户端只负责终端界面、键盘输入、发送队列和接收服务端消息；服务端负责网络连接管理、文档权威状态、操作排序、OT 变换、广播、保存和日志。这样划分后，系统中最关键的一致性逻辑集中在服务端，便于通过单元测试验证。

系统主要模块如下：

- `protocol.py`: 封装 JSON Lines 消息的编码、解码和 socket 收发。
- `document.py`: 实现权威文档、操作模型、OT 变换、版本历史和重同步判断。
- `server.py`: 实现 TCP 服务端、客户端线程、消息分发、广播和自动保存进程管理。
- `client_state.py`: 实现客户端同步状态、待发送队列、ACK 处理和远程操作应用。
- `client.py`: 实现 curses 终端界面、光标移动、按键处理和网络连接。
- `autosave.py`: 实现自动保存、手动保存、备份和日志写入工作进程。
- `tests/`: 包含文档、客户端状态、服务端输出和自动保存相关单元测试。

4.2 网络协议设计

网络协议采用 JSON Lines，即每一条消息都是一个 JSON 对象，并以换行符结束。这种格式便于调试，也便于处理 TCP 粘包和拆包问题。接收端维护字节缓冲区，持续读取 socket 数据，直到遇到换行符后再解码一条完整消息。

典型的编辑操作消息包含消息类型、客户端标识、操作编号、基准版本和操作内容。例如：

```
1 {  
2   "type": "OP",  
3   "client_id": "u1",  
4   "op_id": "u1-12",  
5   "base_version": 8,  
6   "op": {  
7     "kind": "insert",  
8     "pos": 5,  
9     "char": "a"  
10  }  
11 }
```

服务端处理成功后会给提交者返回 `ACK`，并向其他客户端广播 `REMOTE_OP`。这两类消息都携带服务端确认后的 `server_version` 和变换后的权威操作。客户端只应用服务端确认的操作，而不把本地预测结果直接作为最终文档。

4.3 文档锁与全局顺序

多客户端并发编辑时，服务端会有多个线程几乎同时收到 `OP` 消息。如果每个线程直接修改文档，就会出现竞争条件。因此文档对象内部使用可重入锁保护临界区。只有持有文档锁的线程才能检查版本、执行 `OT`、分配版本号、修改文本和写入历史。

这里的设计重点是：服务端不是按照客户端时间戳决定顺序，也不是按照网络包发出时间决定顺序，而是按照操作进入文档临界区的顺序决定全局顺序。进入临界区后分配的递增 `server_version` 就是最终权威顺序。这样可以避免分布式时间带来的不确定性。

另一方面，`socket` 发送和广播不能放在文档锁内部。如果某个客户端网络很慢，持锁发送会导致其他客户端的编辑操作无法进入临界区，扩大阻塞范围。因此服务端在文档锁内只生成 `ACK` 和广播消息，返回后再执行 `socket I/O`。

4.4 OT 位置变换规则

本项目使用字符级 `OT`，因此变换规则集中在位置调整上。当前操作会依次与它的 `base_version` 之后已经发生的历史操作进行比较：

- 历史操作是插入时，当前插入或删除如果位于该插入位置之后或相同位置，就需要右移一位。

- 历史操作是删除时，当前插入如果位于删除位置之后，需要左移一位。
- 历史操作是删除时，当前删除如果位于删除位置之后，需要左移一位。
- 两个删除操作删除同一位置时，后一个删除变为 `noop`，因为目标字符已经被删除。

服务端还会对最终操作做规范化处理，例如插入位置会被限制在文档长度范围内，越界删除会变为 `noop`。这能提高协议容错性，避免异常客户端破坏服务端文档状态。

4.5 客户端状态机

客户端状态机的核心是 `pending_queue` 和 `inflight_op`。用户输入不会直接阻塞等待网络响应，而是先进入队列；发送函数只在当前没有未确认操作时才取出队首并发送。每条操作发送时会记录当前客户端已知版本作为 `base_version`。

客户端收到 ACK 后，会检查 ACK 的 `op_id` 是否对应当前 `inflight_op`。如果匹配，就应用服务端返回的权威操作，更新版本号，清空 `inflight_op`，再继续发送队列中的下一个操作。客户端收到远程操作时，也会检查版本是否连续。如果发现远程版本跳跃，说明中间可能漏收消息，于是请求完整文档。

4.6 自动保存进程

自动保存和日志写入被放在独立进程中完成。主服务端进程处理网络和文档同步，自动保存进程负责磁盘 I/O。二者通过 `multiprocessing.Queue` 传递事件。服务端每处理完一次成功编辑，就发送最新快照；工作进程只保留最新内容，并按固定间隔写入 `.autosave` 文件。

手动保存时，服务端向队列发送 `SAVE_NOW` 事件。工作进程先复制旧文件作为 `.bak` 备份，再使用临时文件替换的方式写入正式文件。这样既体现了进程间通信，也减少了主服务端在网络请求处理过程中直接进行磁盘写入的时间。

5 测试与验证

5.1 单元测试

项目使用 Python 标准库 `unittest` 进行自动化测试。实际运行时，可以在 Linux 服务器克隆 `code` 分支后，进入克隆得到的代码仓库根目录执行测试命令：

```
1 python3 -m unittest discover -s tests -v
```

本地执行结果显示，共运行 17 个测试用例，所有测试均通过，输出结尾为 `Ran 17 tests ... OK`。

测试覆盖的关键场景包括：

- `server_version` 严格递增，保证服务端全局顺序明确。
- 基于旧版本的删除操作经过历史插入变换后，删除位置正确右移。
- 两个客户端删除同一字符时，后到达的删除被转换为 `noop`。
- 历史记录裁剪后，过旧 `base_version` 会触发重同步响应，并返回完整文档状态。
- 客户端连续输入时，只发送一个未确认操作，后续操作留在 `pending_queue` 中等待 ACK。
- 客户端发现远程版本不连续时，会发送 `REQUEST_DOC` 请求完整文档。
- 中文字符可以被插入，终端显示宽度辅助函数能正确处理宽字符。
- 手动保存时会在覆盖正式文件前生成备份文件。

```
root@izbp1la51jwdv4pjbv2r1o2 ~/Linux_Class (code)# python3 -m unittest discover -s tests -v
test_manual_save_writes_backup (test_autosave.AutosaveTests.test_manual_save_writes_backup) ... ok
test_chinese_character_is_insertable (test_client_state.ClientSyncStateTests.test_chinese_character_is_insertable) ... ok
test_chinese_display_width_helpers (test_client_state.ClientSyncStateTests.test_chinese_display_width_helpers) ... ok
test_pending_queue_sends_next_only_after_ack (test_client_state.ClientSyncStateTests.test_pending_queue_sends_next_only_after_ack) ... ok
test_remote_gap_requests_document (test_client_state.ClientSyncStateTests.test_remote_gap_requests_document) ... ok
test_welcome_updates_server_assigned_client_id (test_client_state.ClientSyncStateTests.test_welcome_updates_server_assigned_client_id) ... ok
test_wide_character_backspace_queues_delete (test_client_state.ClientSyncStateTests.test_wide_character_backspace_queues_delete) ... ok
test_delete_position_transforms_after_prior_inserts (test_document.CollaborativeDocumentTests.test_delete_position_transforms_after_prior_inserts) ... ok
test_history_expiry_requests_resync (test_document.CollaborativeDocumentTests.test_history_expiry_requests_resync) ... ok
test_insert_chinese_character (test_document.CollaborativeDocumentTests.test_insert_chinese_character) ... ok
test_same_character_delete_becomes_noop (test_document.CollaborativeDocumentTests.test_same_character_delete_becomes_noop) ... ok
test_server_version_is_strictly_incremented (test_document.CollaborativeDocumentTests.test_server_version_is_strictly_incremented) ... ok
test_format_delete_operation_event (test_server.ServerOutputTests.test_format_delete_operation_event) ... ok
test_format_insert_operation_event (test_server.ServerOutputTests.test_format_insert_operation_event) ... ok
test_format_join_leave_events (test_server.ServerOutputTests.test_format_join_leave_events) ... ok
test_format_save_event (test_server.ServerOutputTests.test_format_save_event) ... ok
test_shutdown_ignores_non_owner_process (test_server.ServerOutputTests.test_shutdown_ignores_non_owner_process) ... ok

-----
Ran 17 tests in 0.004s

OK
```

图 6 测试结果

5.2 本地验证

手动验证用于在不依赖公网、域名解析和安全组规则的情况下，先确认协作编辑器本身的核心功能是否正常。验证时在同一台 Linux 服务器上运行：一个终端运行服务端，同时使用 `tmux` 打开两个客户端窗口进行对比观察。首先在服务端终端中监听本机地址并指定协作编辑文件：

```
1 python3 server.py --host 127.0.0.1 --port 8765 --file shared.py
```

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/Linux_Class (code)# python3 server.py --host 127.0.0.1 --port 8765
--file shared.py
Collaborative server listening on 127.0.0.1:8765
Document: shared.py
JOIN client=sputnik address=('127.0.0.1', 58768)
OP v1 client=sputnik delete pos=129 base=0
OP v2 client=sputnik delete pos=128 base=1
OP v3 client=sputnik delete pos=127 base=2
OP v4 client=sputnik delete pos=126 base=3
OP v5 client=sputnik delete pos=125 base=4
OP v6 client=sputnik delete pos=124 base=5
OP v7 client=sputnik delete pos=123 base=6
OP v8 client=sputnik delete pos=122 base=7
OP v9 client=sputnik delete pos=121 base=8
OP v10 client=sputnik delete pos=120 base=9
OP v11 client=sputnik delete pos=119 base=10
JOIN client=glance address=('127.0.0.1', 46514)
OP v12 client=glance insert pos=119 char='\n' base=11
OP v13 client=glance insert pos=120 char='\n' base=12
OP v14 client=glance insert pos=121 char='a' base=13
OP v15 client=glance insert pos=122 char='d' base=14
OP v16 client=glance insert pos=123 char='a' base=15
OP v17 client=glance insert pos=124 char='s' base=16
OP v18 client=glance insert pos=125 char='d' base=17
```

图 7 本地运行服务端界面

从服务端终端输出也可以看到，两个客户端加入时都会打印 `JOIN` 事件，并且地址字段中的 IP 均为 `127.0.0.1`。这说明 `sputnik` 和 `glance` 两个客户端虽然分别运行在不同的 `tmux` 窗口中，但网络连接都通过本机回环接口进入同一个服务端进程，符合本地手动验证的设计。

如果启动时出现 `Address already in use`，说明 `8765` 端口已经被已有进程占用，通常是前一次启动的服务端仍在运行。此时可以直接使用已有服务端继续启动客户端进行验证；如果需要重新启动服务端，则应先停止原服务端进程，或者临时换用其他端口并让两个客户端使用相同的新端口连接。

随后在 `tmux` 中打开两个客户端窗口或窗格，两个客户端都连接同一个本机服务端，但使用不同的 `client_id`：

```
1 python3 client.py --host 127.0.0.1 --port 8765 --client-id sputnik
2 python3 client.py --host 127.0.0.1 --port 8765 --client-id glance
```

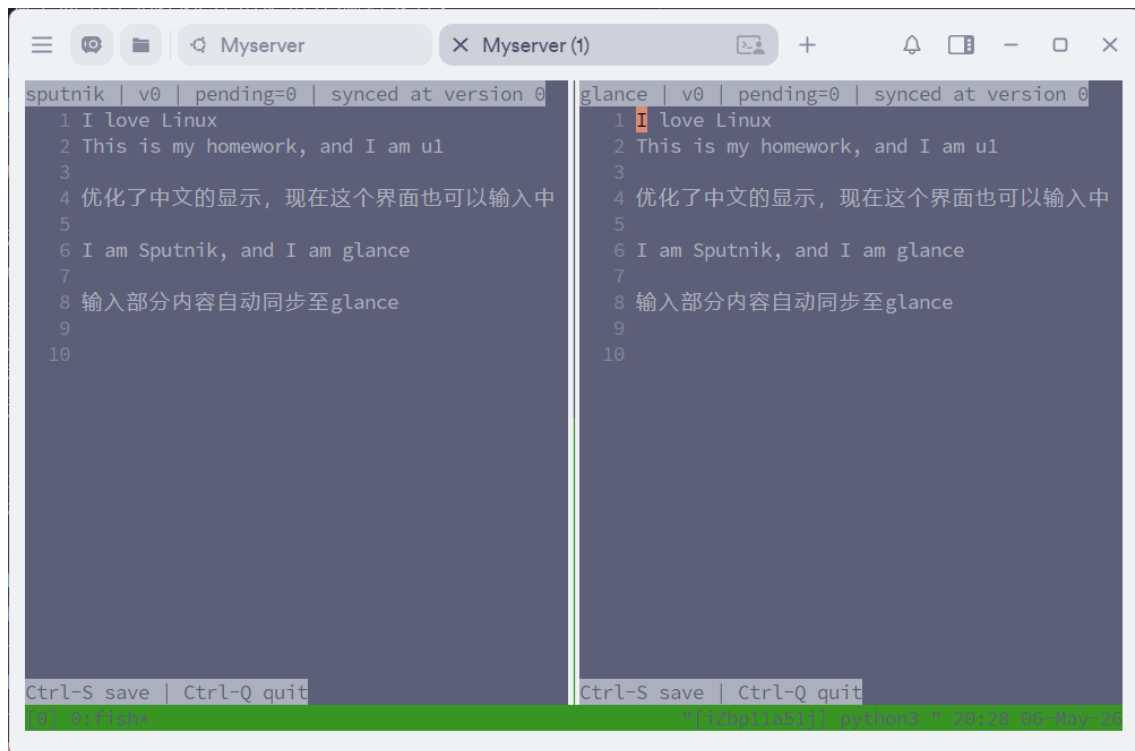



图 8 tmux 中两个本地客户端同步显示

图中左侧客户端使用 `sputnik`，右侧客户端使用 `glance`。两个客户端的状态栏都显示当前处于同步状态，文档内容也保持一致。实际验证时，在任意一个客户端输入英文、中文、换行或删除内容后，另一个客户端都会同步显示相同修改，说明本机环境下的消息发送、ACK 确认、服务端广播和客户端文档更新流程能够正常工作。

进一步验证异常情况时，可以在服务端终端中主动结束服务端进程。服务端关闭后，两个客户端不会继续显示为同步状态，而是在状态栏中显示 `error: connection closed`，提示与服务端的 TCP 连接已经断开；同时客户端中已经同步到的文档内容仍然保留在界面上，便于观察断线前的最终状态。

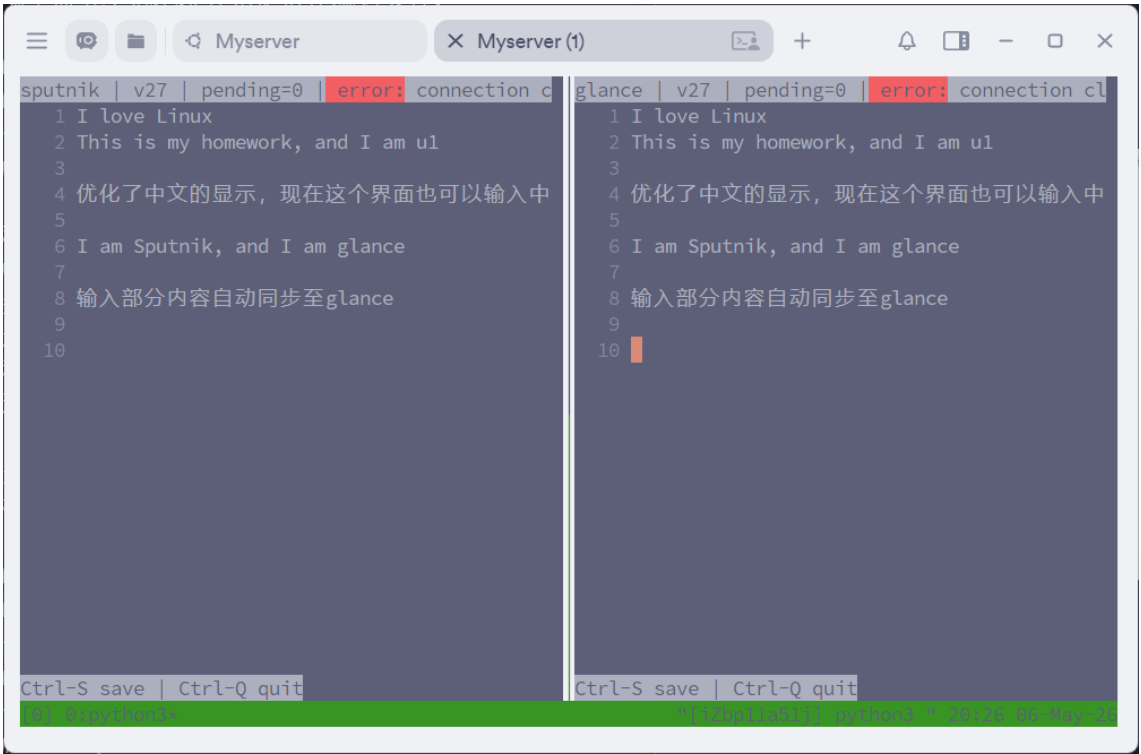


图 9 服务端关闭后客户端显示连接断开

5.3 部署验证

部署验证主要围绕真实公网运行流程展开。由于服务端运行在阿里云 ECS 服务器上, 要让外部客户端能够访问该协作编辑服务, 需要先在阿里云安全组中开放服务端使用的 8765 端口。只有安全组规则允许公网访问后, 客户端才能通过 `glance02.xyz:8765` 连接到服务端。

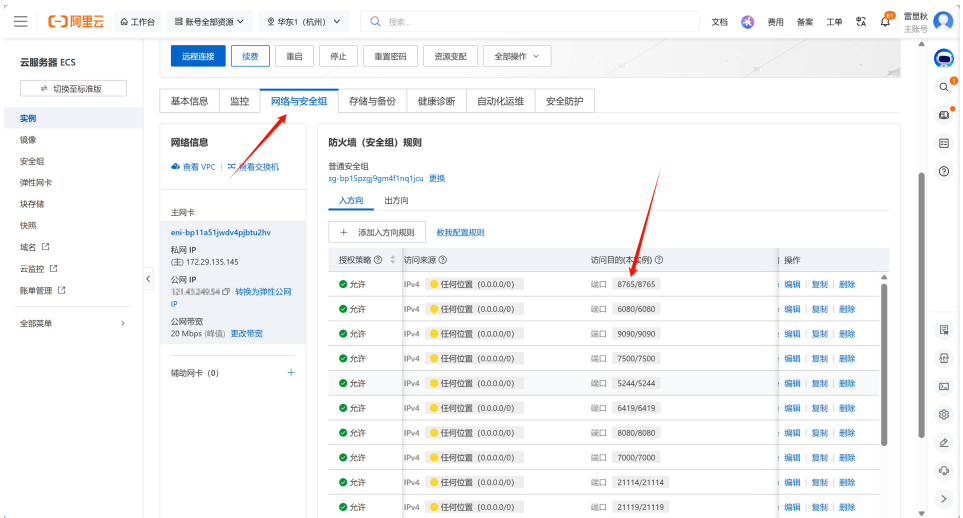


图 10 阿里云开放 8765 端口

完成端口放行后，在 Linux 服务器上启动服务端，监听 `0.0.0.0:8765`，并指定协作编辑文件。由于服务器已经绑定域名 `glance02.xyz`，客户端可以直接通过该域名连接：

```
1 python3 client.py --host glance02.xyz --port 8765 --client-id u1
```

随后在不同终端中启动两个客户端，分别使用不同 `client_id`。当其中一个客户端输入字符或换行时，另一个客户端会收到 `REMOTE_OP` 并同步显示更新。服务端终端会输出类似 `JOIN`、`OP v...`、`SAVE requested`、`LEAVE` 的事件日志，便于观察连接、编辑和保存流程。

下面是客户端 `sputnik` 连续输入换行和 `I am Sput` 时，服务端终端打印出的操作序列。可以看到每一次按键都会形成一条字符级 `insert` 操作，`v1` 到 `v10` 是服务端分配的全局递增版本号，`base` 则是客户端发送该操作时已确认的版本号。

```
1 OP v1 client=sputnik insert pos=70 char='\n' base=0
2 OP v2 client=sputnik insert pos=71 char='I' base=1
3 OP v3 client=sputnik insert pos=72 char=' ' base=2
4 OP v4 client=sputnik insert pos=73 char='a' base=3
5 OP v5 client=sputnik insert pos=74 char='m' base=4
6 OP v6 client=sputnik insert pos=75 char=' ' base=5
7 OP v7 client=sputnik insert pos=76 char='S' base=6
8 OP v8 client=sputnik insert pos=77 char='p' base=7
9 OP v9 client=sputnik insert pos=78 char='u' base=8
10 OP v10 client=sputnik insert pos=79 char='t' base=9
```

这段日志能够直接反映 ACK 驱动发送队列的效果：客户端收到上一条操作的 ACK 后才继续发送下一条，所以 `base` 会随着服务端版本逐步递增。另一方面，`pos` 也连续增加，说明服务端按字符位置把换行和后续英文字符顺序应用到了同一份权威文档中。

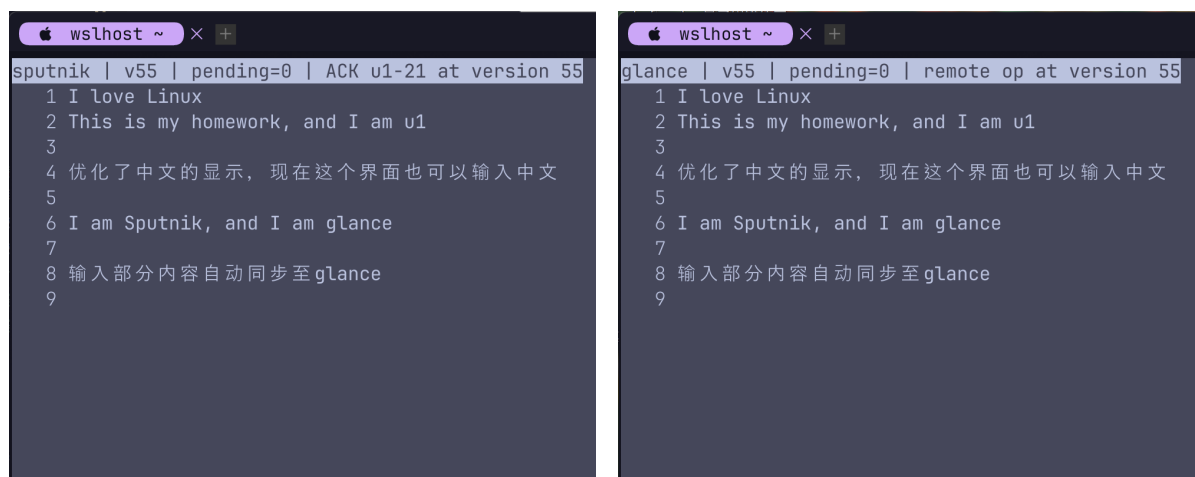


图 11 客户端 `sputnik` 输入内容后客户端 `glance` 自动同步显示

```

root@iZbp1la51jwdv4pjbv2r1oZ ~/Linux_Class (code)# python3 server.py --host 0.0.0.0 --port 8765 --file shared.py
Collaborative server listening on 0.0.0.0:8765
Document: shared.py
JOIN client=sputnik address=('182.96.176.182', 2291)
JOIN client=glance address=('36.156.65.4', 15875)
OP v1 client=sputnik insert pos=120 char='\n' base=0
OP v2 client=sputnik insert pos=121 char='测' base=1
OP v3 client=sputnik insert pos=122 char='试' base=2
OP v4 client=sputnik insert pos=123 char='部' base=3
OP v5 client=sputnik insert pos=124 char='分' base=4
LEAVE client=sputnik
LEAVE client=glance
^CSAVE shutdown version=5 file=shared.py

```

图 12 服务端终端输出 JOIN、OP、SAVE、LEAVE 等事件日志

为了验证 OT 行为，可以构造两个客户端基于不同版本编辑同一位置的场景。例如一个客户端连续在某个位置插入字符，另一个客户端基于旧版本删除该位置字符。根据服务端日志中的 `transform_trace`，可以观察到操作位置在应用前被转换。最终两个客户端显示的文档内容保持一致。



```

root@iZbp1la51jwdv4pjbv2r1oZ ~/Linux_Class (code)# tail -n 50 shared.py.edit.log | jq 'select(.op_id=="stale-transform")'
{
  "base_version": 0,
  "client_id": "stale",
  "content_length": 130,
  "event": "operation",
  "op_id": "stale-transform",
  "original_op": {
    "char": "|日",
    "kind": "insert",
    "pos": 120
  },
  "server_version": 5,
  "timestamp": 1777911047.3625264,
  "transform_trace": [
    {
      "after": {
        "char": "|日",
        "kind": "insert",
        "pos": 121
      },
      "against_op": {
        "char": "|日",
        "kind": "insert",
        "pos": 120
      },
      "against_version": 1,
      "before": {
        "char": "|日",
        "kind": "insert",
        "pos": 120
      }
    }
  ]
}

```

图 13 编辑日志中的 `transform_trace` 位置变换记录

图中将原始 JSON 日志中的 `transform_trace` 提取为位置变换过程。该操作的 `op_id` 是 `stale-transform`，提交时基于旧版本 0，并在服务端版本 5 被确认。操作原本希望在位置 120 插入字符，服务端根据之后已经发生的历史插入操作依次调整位置，最终将其应用到位置 124。

表 1 `transform_trace` 位置变换摘要

<code>against_version</code>	<code>against_op</code>	<code>before_pos</code> → <code>after_pos</code>
1	<code>insert(pos=120, char=旧)</code>	120 → 121
2	<code>insert(pos=121, char=A)</code>	121 → 122
3	<code>insert(pos=122, char=B)</code>	122 → 123
4	<code>insert(pos=123, char=C)</code>	123 → 124

该验证说明项目不仅可以在本机 `127.0.0.1` 上运行，也可以在真实网络环境中作为远程协作工具使用。

6 问题与解决方案

6.1 并发操作顺序不确定

问题：多个客户端线程可能同时提交编辑操作。如果服务端没有统一排序，不同线程对文档的修改顺序可能不稳定，客户端最终内容也可能不一致。

解决方案：将权威文档状态集中在服务端，并在 `CollaborativeDocument` 内部使用文档锁。每个操作进入锁保护的临界区后才分配 `server_version`。因此全局顺序由服务端临界区顺序决定，而不是由客户端时间或网络延迟决定。

6.2 旧版本操作位置失效

问题：客户端提交操作时使用的是本地版本。如果该操作到达服务端时，其他客户端已经修改过文档，那么原始位置可能已经不再正确。

解决方案：服务端保存有限历史。处理旧版本操作时，从 `base_version` 之后的历史操作开始逐条执行 OT 位置变换，将操作转换到当前版本后再应用。这样旧版本操作仍然能表达用户原本想编辑的位置。

6.3 连续输入容易丢失或乱序

问题：用户在终端中快速输入多个字符时，如果客户端每次按键都立即发送，同时又没有处理未确认操作，容易造成本地版本混乱。

解决方案：客户端使用 `pending_queue` 保存所有输入，并限制同一时刻最多一个 `inflight_op`。只有收到当前操作的 ACK 后，才发送下一个操作。这样牺牲了一点发

送并行度，但显著降低了客户端同步复杂度。

6.4 慢客户端可能阻塞服务端

问题：如果服务端在文档锁内向所有客户端广播消息，而某个客户端网络很慢，socket 发送可能阻塞，从而导致其他客户端无法继续提交编辑操作。

解决方案：文档锁内只完成状态修改和消息构造，ACK 和广播都放在锁外执行。广播时也只短暂持有客户端列表锁来复制收件人列表，真正发送时不再持有该锁。

6.5 历史无限增长

问题：如果服务端永久保存所有操作历史，长时间运行后内存占用会持续增加。

解决方案：历史记录使用有限长度队列保存最近的操作，最大条数由代码中的历史限制参数控制。同时维护 `history_start_version`，记录当前历史能够支持的最早基准版本。当客户端版本太旧，已经无法根据保留历史进行 OT 时，服务端返回 `RESYNC_REQUIRED` 并发送完整文档。

6.6 文件保存可能覆盖旧内容

问题：手动保存会覆盖正式文件，如果误操作或程序异常，可能丢失旧版本内容。

解决方案：保存前生成 `.bak` 备份文件，并使用临时文件替换的方式进行原子写入。自动保存则写入独立的 `.autosave` 文件，不直接覆盖正式文件。

6.7 中文字符显示宽度

问题：终端中中文字符通常占两个显示列，如果简单按照字符串长度移动光标，显示位置会错乱。

解决方案：客户端使用 Python 的 `unicodedata` 模块，通过 `east_asian_width` 判断宽字符，并提供显示列和字符列之间的转换函数。测试中也包含中文字符插入和显示宽度计算场景。

7 项目不足与改进方向

虽然 TermSync 已经能够完成多人连接、实时同步、保存、自动保存、日志记录和测试验证，但它仍然是一个课程项目规模的协作编辑器，和成熟协作工具相比还存在

一些不足。首先，当前编辑粒度是字符级操作，便于说明 OT 原理，但快速输入大量文本时会产生较多网络消息；后续可以考虑把连续输入合并为批量操作。其次，系统暂时没有加入用户认证和权限控制，只适合可信网络或课堂演示场景，如果要公开部署，需要补充连接鉴权、访问控制和异常客户端隔离。再次，客户端的终端界面功能较基础，还可以继续增加状态栏、只读提示、断线重连和冲突恢复提示，使使用体验更加完整。

从实现角度看，后续还可以继续扩展测试范围。例如增加更复杂的并发插入、插入与删除交错、历史裁剪后客户端重连等集成测试；在服务端加入更细粒度的运行日志和错误统计；在保存模块中加入异常注入测试，验证磁盘写入失败时备份文件和自动保存文件是否仍然可靠。这些改进能够进一步提升项目的稳定性，也能让课程报告中的系统设计更加接近真实 Linux 服务程序。

8 代码展示

本文所对应的项目代码托管于 GitHub 仓库 [glance02/Linux_Class](#)。其中，程序实现位于 `code` 分支，论文正文的 LaTeX 源码位于 `master` 分支。项目完整实现包含服务端、客户端、协议封装、持久化、日志和测试等多个模块，代码量较大；受篇幅限制，本节仍以关键代码片段为主。下文代码展示中涉及的文件路径，均以服务器克隆 `code` 分支后得到的代码仓库根目录为基准。

8.1 协议与基础数据结构

协议封装：协作编辑首先需要稳定的消息边界和统一的数据表示。项目使用 JSON Lines 作为 TCP 字节流上的应用层协议：发送时把 Python 字典编码为 UTF-8 JSON，并在末尾追加换行符；接收时先在缓冲区中寻找换行符，再解码出完整消息。协议封装还为发送操作加锁，避免同一连接被多个线程同时写入时出现字节交叉。

```
1 """ 协作编辑器使用的 JSON Lines 协议辅助函数。
2
3 TCP 是字节流，不会天然保留“消息边界”。本项目把每条 JSON 消息后面加
4 一个换行符，形成 JSON Lines 协议，这样接收方只要读到换行就得到一条完整消息。
5 """
6
7 from __future__ import annotations
8
9 import json
10 import socket
11 import threading
```

```
12 from typing import Any, Dict
13
14
15 Message = Dict[str, Any]
16 # 单条消息设置上限, 避免异常客户端一直发送不带换行的大数据撑爆内存。
17 MAX_LINE_BYTES = 1024 * 1024
18
19
20 def encode_message(message: Message) -> bytes:
21     """ 把一条 JSON 消息编码为 UTF-8 的 JSON Lines 数据帧。"""
22     # ensure_ascii=False 让日志和协议里可以直接看到中文字符,
23     # separators 去掉多余空格, 减少网络传输字节数。
24     return (
25         json.dumps(message, ensure_ascii=False, separators=(",",
26             ↪ ": ")).encode("utf-8")
27         + b"\n"
28     )
29
30 def decode_message(line: bytes) -> Message:
31     """ 把一条 JSON Lines 数据帧解码为消息字典。"""
32     if not line:
33         raise EOFError("connection closed")
34     try:
35         payload = json.loads(line.decode("utf-8"))
36     except json.JSONDecodeError as exc:
37         raise ValueError(f"invalid JSON message: {exc}") from exc
38     if not isinstance(payload, dict):
39         # 协议顶层必须是对象, 方便所有消息都用 type 字段分发。
40         raise ValueError("protocol message must be a JSON object")
41     return payload
42
43
44 class JsonLineSocket:
45     """ 对套接字做一层轻量封装, 用于收发按换行分隔的 JSON 消息。"""
46
47     def __init__(self, sock: socket.socket):
48         self.sock = sock
49         # recv 可能一次读到半条消息, 也可能读到多条消息。
50         # _buffer 保存尚未被 decode 的字节, 直到找到换行符。
51         self._buffer = bytearray()
52         self._send_lock = threading.Lock()
53
54     def recv_message(self) -> Message:
55         while True:
56             newline_index = self._buffer.find(b"\n")
57             if newline_index >= 0:
```

```

58         # 找到一条完整 JSON Lines 消息后, 只消费到换行符为止;
59         # 换行后面的字节保留给下一次 recv_message。
60         line = bytes(self._buffer[:newline_index])
61         del self._buffer[: newline_index + 1]
62         return decode_message(line)
63
64     chunk = self.sock.recv(4096)
65     if not chunk:
66         raise EOFError("connection closed")
67     self._buffer.extend(chunk)
68     if len(self._buffer) > MAX_LINE_BYTES:
69         raise ValueError("protocol message exceeds maximum line length")
70
71     def send_message(self, message: Message) -> None:
72         data = encode_message(message)
73         # 同一个客户端连接可能被读线程、UI 线程或服务端广播逻辑同时使用。
74         # 这里用发送锁保证一条 JSON Lines 消息会完整写入套接字,
75         # 避免两次 sendall 的字节交叉在一起, 导致对端 JSON 解析失败。
76         with self._send_lock:
77             self.sock.sendall(data)

```

操作模型: 编辑操作、历史记录和处理结果集中定义在 `document.py` 中。`Operation` 表示字符级 `insert`、`delete` 和 `noop`, 并在从网络消息解析时进行类型检查; `HistoryEntry` 保存已经进入服务端全局顺序的历史操作; `ProcessResult` 则把文档处理结果拆成 `ACK`、广播消息、日志和快照, 方便服务端在锁外执行网络发送和磁盘保存。

```

1  """ 带有服务端 OT 和有限历史记录权威文档模型。
2
3  这个模块不关心 socket、终端界面或文件保存, 只维护一份服务端权威文本。
4  所有客户端提交的编辑操作都会先进入这里, 经过校验、OT 位置变换和版本分配后,
5  才会变成可以广播给所有客户端的权威操作。
6  """
7
8  from __future__ import annotations
9
10 from collections import deque
11 from dataclasses import dataclass, field
12 from threading import RLock
13 from typing import Any, Deque, Dict, Iterable, List, Optional
14
15
16 VALID_KINDS = {"insert", "delete", "noop"}
17
18
19 @dataclass

```



```

20 class Operation:
21     """ 单个字符级编辑操作。
22
23     kind 表示操作类型; pos 使用 Python 字符串索引; insert 需要 char,
24     delete 不需要 char; noop 用来表示“这个操作被历史操作抵消了”。
25     """
26
27     kind: str
28     pos: int = 0
29     char: Optional[str] = None
30     reason: Optional[str] = None
31
32     @classmethod
33     def from_dict(cls, payload: Dict[str, Any]) -> "Operation":
34         # 网络协议传来的 JSON 字典不能直接信任, 先在边界处统一校验。
35         # 校验通过后, 后续文档逻辑就可以按 Operation 对象处理。
36         if not isinstance(payload, dict):
37             raise ValueError("operation must be a JSON object")
38         kind = payload.get("kind")
39         if kind not in VALID_KINDS:
40             raise ValueError(f"unsupported operation kind: {kind!r}")
41         pos = payload.get("pos", 0)
42         if not isinstance(pos, int):
43             raise ValueError("operation pos must be an integer")
44         char = payload.get("char")
45         if kind == "insert":
46             if not isinstance(char, str) or len(char) != 1:
47                 raise ValueError("insert operation requires a single-character char")
48         else:
49             char = None
50         reason = payload.get("reason")
51         if reason is not None and not isinstance(reason, str):
52             reason = str(reason)
53         return cls(kind=kind, pos=pos, char=char, reason=reason)
54
55     def clone(self) -> "Operation":
56         # OT 变换会调整位置, 为了保留 original_op 用于日志和 ACK,
57         # 这里总是复制出一份新对象再修改。
58         return Operation(self.kind, self.pos, self.char, self.reason)
59
60     def to_dict(self) -> Dict[str, Any]:
61         # 协议消息和日志都使用普通 dict, 方便 JSON 序列化。
62         payload: Dict[str, Any] = {"kind": self.kind, "pos": self.pos}
63         if self.char is not None:
64             payload["char"] = self.char
65         if self.reason:
66             payload["reason"] = self.reason

```

```

67     return payload
68
69     def as_noop(self, reason: str) -> "Operation":
70         # 当操作无法再产生实际效果时，不直接丢弃，而是记录成 noop。
71         # 这样服务端版本号仍然递增，客户端也能收到明确 ACK。
72         return Operation("noop", self.pos, None, reason)
73
74
75 @dataclass
76 class HistoryEntry:
77     """ 服务端已经接受并排序的一条权威历史操作。 """
78
79     server_version: int
80     client_id: str
81     op_id: str
82     base_version: int
83     original_op: Operation
84     op: Operation
85
86     def to_dict(self) -> Dict[str, Any]:
87         # 主要用于调试或未来扩展日志；当前核心流程直接访问字段。
88         return {
89             "server_version": self.server_version,
90             "client_id": self.client_id,
91             "op_id": self.op_id,
92             "base_version": self.base_version,
93             "original_op": self.original_op.to_dict(),
94             "op": self.op.to_dict(),
95         }
96
97
98 @dataclass
99 class ProcessResult:
100     """ 文档处理一次客户端操作后的结果包。
101
102     文档模型只负责计算结果，不直接发送 socket 或写文件。
103     调用方根据这里的 ACK、REMOTE_OP、日志和快照决定后续副作用。
104     """
105
106     status: str
107     messages: List[Dict[str, Any]] = field(default_factory=list)
108     ack: Optional[Dict[str, Any]] = None
109     remote: Optional[Dict[str, Any]] = None
110     log_event: Optional[Dict[str, Any]] = None
111     snapshot_content: Optional[str] = None
112     snapshot_version: Optional[int] = None

```

8.2 服务端权威文档与 OT

文本应用与位置变换：文本修改函数负责把规范化后的操作真正作用到字符串上，而 OT 变换函数负责根据历史操作修正位置。当一个客户端操作基于旧版本提交时，服务端会把它依次与之后已经发生的历史操作比较，修正位置偏移；如果两个删除操作指向同一个字符，后到达的删除会变为 `noop`，从而避免误删后续字符。

```

115 def apply_operation_to_text(text: str, op: Operation) -> str:
116     """ 把已经规范化的位置操作应用到文本上。 """
117     if op.kind == "noop":
118         return text
119     if op.kind == "insert":
120         pos = max(0, min(op.pos, len(text)))
121         return text[:pos] + (op.char or "") + text[pos:]
122     if op.kind == "delete":
123         if 0 <= op.pos < len(text):
124             return text[: op.pos] + text[op.pos + 1 :]
125         return text
126     raise ValueError(f"unsupported operation kind: {op.kind!r}")
127
128
129 def transform_against(op: Operation, previous: Operation) -> Operation:
130     """ 根据一条已经发生的历史操作，变换当前操作的位置。 """
131     result = op.clone()
132     # noop 不改变文本，也不会影响其他操作的位置。
133     if result.kind == "noop" or previous.kind == "noop":
134         return result
135
136     if previous.kind == "insert":
137         # 历史中已经发生过一次插入，会把它后面的字符整体向右推一格。
138         # 因此当前操作如果作用在插入点之后，或者正好作用在同一位置，
139         # 都需要把位置加 1，才能继续指向原来想编辑的字符。
140         if result.kind == "insert" and previous.pos <= result.pos:
141             result.pos += 1
142         elif result.kind == "delete" and previous.pos <= result.pos:
143             result.pos += 1
144         return result
145
146     if previous.kind == "delete":
147         # 历史中已经发生过一次删除，会把它后面的字符整体向左拉一格。
148         # 插入操作只在历史删除点严格位于自己之前时左移；如果同位置插入，
149         # 插入仍然发生在这个位置。删除操作如果删除同一个字符，则变成 noop。
150         if result.kind == "insert" and previous.pos < result.pos:
151             result.pos -= 1
152         elif result.kind == "delete":

```

```

153         if previous.pos < result.pos:
154             result.pos -= 1
155         elif previous.pos == result.pos:
156             return result.as_noop("same character already deleted")
157         return result
158
159     raise ValueError(f"unsupported history operation kind: {previous.kind!r}")
160

```

权威文档处理流程：`CollaborativeDocument` 是服务端的权威文档对象。它使用 `RLock` 保护文本、版本号和有限历史记录，所有编辑操作只有进入该临界区后才会分配递增的服务端版本号。`process_operation` 展示了完整处理流程：解析操作、检查版本、执行 OT、规范化位置、应用文本、写入历史，并生成后续需要发送或保存的结果。

```

162 class CollaborativeDocument:
163     """ 线程安全的服务端权威文档状态。
164
165     编辑操作只有进入当前对象的锁保护临界区后，才会获得服务端版本号。
166     这个版本号也就是该操作在有限历史记录中的全局顺序编号。
167     """
168
169     def __init__(self, initial_text: str = "", history_limit: int = 1000):
170         if history_limit <= 0:
171             raise ValueError("history_limit must be positive")
172         self._text = initial_text
173         # version 是服务端已经接受的操作数量，也是最新 DOC_STATE 的版本号。
174         self.version = 0
175         # history 只保留最近一段操作，用于把旧客户端操作变换到当前版本。
176         # deque(maxlen=...) 会自动丢弃最老的记录。
177         self.history: Deque[HistoryEntry] = deque(maxlen=history_limit)
178         # history_start_version 表示仍可支持 OT 的最早 base_version。
179         self.history_start_version = 0
180         # RLock 保护文本、版本号和历史记录，保证多个客户端线程不会交叉修改。
181         self._lock = RLock()
182
183     def snapshot(self) -> Dict[str, Any]:
184         # 返回完整文档状态，用于新客户端加入或客户端版本断档后的重同步。
185         with self._lock:
186             return {
187                 "type": "DOC_STATE",
188                 "version": self.version,
189                 "history_start_version": self.history_start_version,
190                 "content": self._text,
191             }
192

```

```

193     def content(self) -> str:
194         # 保存线程需要读取当前文本。这里同样走锁，避免读到更新到一半的状态。
195         with self._lock:
196             return self._text
197
198     def process_operation(
199         self,
200         client_id: str,
201         op_id: str,
202         base_version: int,
203         op_payload: Dict[str, Any],
204     ) -> ProcessResult:
205         # 第一层校验在锁外完成，避免明显非法请求占用文档临界区。
206         try:
207             original_op = Operation.from_dict(op_payload)
208         except ValueError as exc:
209             return ProcessResult(
210                 status="error",
211                 messages=[{"type": "ERROR", "op_id": op_id, "message": str(exc)}],
212             )
213
214         with self._lock:
215             # 下面这段就是服务端的“文档处理临界区”：
216             # 1. 检查客户端版本是否还能被历史覆盖；
217             # 2. 对旧版本操作做 OT 位置变换；
218             # 3. 分配新的 server_version；
219             # 4. 应用操作并写入有限历史。
220             # 套接字发送不在这里做，避免慢客户端长期占用文档锁。
221             if not isinstance(base_version, int):
222                 return ProcessResult(
223                     status="error",
224                     messages=[
225                         {
226                             "type": "ERROR",
227                             "op_id": op_id,
228                             "message": "base_version must be an integer",
229                         }
230                     ],
231                 )
232
233             if base_version < self.history_start_version or base_version >
234                 ↪ self.version:
235                 # base_version 太旧：服务端已没有足够历史可做 OT。
236                 # base_version 太新：客户端声称见过服务端还没有产生的版本。
237                 # 两种情况都不能安全推断位置，只能要求客户端重同步完整文档。
238                 return ProcessResult(
239                     status="resync",

```

```

239         messages=[
240             {
241                 "type": "RESYNC_REQUIRED",
242                 "op_id": op_id,
243                 "reason": "operation history expired"
244                 if base_version < self.history_start_version
245                 else "client version is ahead of server",
246                 "server_version": self.version,
247                 "history_start_version": self.history_start_version,
248             },
249             self._snapshot_unlocked(),
250         ],
251     )
252
253     transformed = original_op.clone()
254     transform_trace: List[Dict[str, Any]] = []
255     # 客户端提交的 base_version 可能落后于服务端当前版本。
256     # 这里把该版本之后的每一条权威历史操作依次拿出来，
257     # 将客户端操作一步步变换到“现在”可以安全应用的位置。
258     for entry in self._history_after_unlocked(base_version):
259         before = transformed.to_dict()
260         transformed = transform_against(transformed, entry.op)
261         after = transformed.to_dict()
262         if before != after:
263             transform_trace.append(
264                 {
265                     "against_version": entry.server_version,
266                     "against_op": entry.op.to_dict(),
267                     "before": before,
268                     "after": after,
269                 }
270             )
271
272     transformed = self._normalize_for_apply_unlocked(transformed)
273     # 操作只有进入临界区并完成 OT 后才获得 server_version。
274     # 这个递增编号就是全局操作顺序，不依赖客户端时间戳或网络发送顺序。
275     self.version += 1
276     server_version = self.version
277     self._text = apply_operation_to_text(self._text, transformed)
278
279     history_entry = HistoryEntry(
280         server_version=server_version,
281         client_id=client_id,
282         op_id=op_id,
283         base_version=base_version,
284         original_op=original_op,
285         op=transformed,

```

```
286     )
287     self.history.append(history_entry)
288     self._refresh_history_start_unlocked()
289
290     # ACK 发给提交者: 告诉它自己的 op_id 已经被服务端接受,
291     # 并给出最终权威位置。REMOTE_OP 发给其他客户端同步同一操作。
292     ack = {
293         "type": "ACK",
294         "client_id": client_id,
295         "op_id": op_id,
296         "base_version": base_version,
297         "server_version": server_version,
298         "version": self.version,
299         "op": transformed.to_dict(),
300         "original_op": original_op.to_dict(),
301     }
302     remote = {
303         "type": "REMOTE_OP",
304         "client_id": client_id,
305         "op_id": op_id,
306         "server_version": server_version,
307         "version": self.version,
308         "op": transformed.to_dict(),
309     }
310     log_event = {
311         # 日志记录原始操作、变换后操作和变换轨迹, 便于报告中展示 OT 过程。
312         "event": "operation",
313         "server_version": server_version,
314         "client_id": client_id,
315         "op_id": op_id,
316         "base_version": base_version,
317         "original_op": original_op.to_dict(),
318         "transformed_op": transformed.to_dict(),
319         "transform_trace": transform_trace,
320         "content_length": len(self._text),
321     }
322     return ProcessResult(
323         status="ok",
324         ack=ack,
325         remote=remote,
326         log_event=log_event,
327         snapshot_content=self._text,
328         snapshot_version=self.version,
329     )
```

重同步辅助逻辑: 当客户端版本太旧时, 服务端需要返回完整文档快照; 当操作位

置越界时，服务端也要在应用前进行规范化。下面的内部辅助函数负责生成快照、筛选历史、规范化操作，并维护当前历史仍能支持的最早 `base_version`。

```

331 def _snapshot_unlocked(self) -> Dict[str, Any]:
332     # 调用者已经持有 _lock 时使用，避免重复加锁造成代码层次混乱。
333     return {
334         "type": "DOC_STATE",
335         "version": self.version,
336         "history_start_version": self.history_start_version,
337         "content": self._text,
338     }
339
340 def _history_after_unlocked(self, base_version: int) -> Iterable[HistoryEntry]:
341     # 只返回客户端 base_version 之后发生的权威操作。
342     # 当前操作需要依次穿过这些历史，才能落在最新文本的正确位置。
343     for entry in self.history:
344         if entry.server_version > base_version:
345             yield entry
346
347 def _normalize_for_apply_unlocked(self, op: Operation) -> Operation:
348     # 经过 OT 后的位置仍然可能越界，例如客户端删除一个已经不存在的位置。
349     # 在真正改文本前统一夹取或转换为 noop，保证 apply 阶段简单可靠。
350     if op.kind == "noop":
351         return op
352     if op.kind == "insert":
353         pos = max(0, min(op.pos, len(self._text)))
354         return Operation("insert", pos, op.char)
355     if op.kind == "delete":
356         if 0 <= op.pos < len(self._text):
357             return Operation("delete", op.pos)
358         return op.as_noop("delete position out of range")
359     raise ValueError(f"unsupported operation kind: {op.kind!r}")
360
361 def _refresh_history_start_unlocked(self) -> None:
362     if not self.history:
363         self.history_start_version = self.version
364         return
365     # 如果当前保留的最早历史操作版本是 N + 1，那么客户端从 base N
366     # 开始仍然可以做 OT 变换；更旧的版本就必须重同步完整文档。
367     # deque(maxlen=N) 会自动丢弃旧历史，所以这里记录最早还能支持
368     # OT 变换的 base_version。客户端再旧就不能猜测位置，只能重同步全文。
369     self.history_start_version = max(0, self.history[0].server_version - 1)

```

8.3 服务端连接、广播与保存触发

事件输出与连接对象：服务端一方面要维护面向用户的运行日志，另一方面要为每个客户端连接保存必要信息。下面的代码展示了终端日志格式化函数和 `ClientConnection`。这些格式化函数不改变协议内容，只把结构化事件转换成便于演示观察的 `JOIN`、`OP`、`SAVE` 等文本。

```

1  """ 服务端权威协作编辑器的 TCP 服务端。
2
3  服务端是整个协作系统中唯一的“事实来源”：客户端只提交自己基于某个
4  版本产生的编辑意图，真正的排序、OT 变换、应用、广播和保存都在这里完成。
5  这样可以把复杂的并发冲突处理集中在服务端，客户端逻辑保持相对简单。
6  """
7
8  from __future__ import annotations
9
10 import argparse
11 import multiprocessing
12 import os
13 import signal
14 import socket
15 import threading
16 from dataclasses import dataclass
17 from pathlib import Path
18 from typing import Dict, List, Optional, Tuple
19
20 from autosave import autosave_worker
21 from document import CollaborativeDocument, ProcessResult
22 from protocol import JsonLineSocket, Message
23
24
25 def format_operation_event(entry: Message) -> str:
26     # 把结构化日志事件转换成人能直接阅读的终端输出。
27     # 这类格式化函数不参与协议，只是为了课堂演示时便于观察服务端行为。
28     op = entry.get("transformed_op", {})
29     kind = op.get("kind", "unknown") if isinstance(op, dict) else "unknown"
30     pos = op.get("pos", "?") if isinstance(op, dict) else "?"
31     message = (
32         f"OP v{entry.get('server_version')} client={entry.get('client_id')} "
33         f"{kind} pos={pos}"
34     )
35     if isinstance(op, dict) and kind == "insert":
36         message += f" char={op.get('char')!r}"
37     message += f" base={entry.get('base_version')}"
38     return message

```

```

39
40
41 def format_server_event(entry: Message) -> Optional[str]:
42     # 服务端日志里可能有多种事件。这里集中维护展示格式，避免业务逻辑里
43     # 到处拼接字符串，也方便单元测试只验证格式化结果。
44     event = entry.get("event")
45     if event == "operation":
46         return format_operation_event(entry)
47     if event == "join":
48         return f"JOIN client={entry.get('client_id')} address={entry.get('address')}"
49     if event == "leave":
50         return f"LEAVE client={entry.get('client_id')}"
51     return None
52
53
54 def format_save_event(reason: str, client_id: Optional[str], version: object, path:
55     ↪ Path) -> str:
56     # 保存可能来自客户端请求，也可能来自服务端关闭时的最终保存。
57     # client_id 为空时表示不是某个具体客户端触发的保存。
58     if client_id:
59         return f"SAVE {reason} client={client_id} version={version} file={path}"
60     return f"SAVE {reason} version={version} file={path}"
61
62 @dataclass
63 class ClientConnection:
64     """ 服务端保存的单个客户端连接信息。
65
66     address 用于日志观察，transport 负责真正的 JSON Lines 收发。
67     """
68
69     client_id: str
70     address: Tuple[str, int]
71     transport: JsonLineSocket
72
73     def send(self, message: Message) -> None:
74         self.transport.send_message(message)
75

```

初始化、监听与关闭：`CollaborativeServer` 在启动时读取协作文档、创建权威文档对象、准备客户端连接表，并启动独立的自动保存进程。监听循环使用 TCP socket 接受客户端连接，每个连接交给一个线程处理；关闭时则向保存进程发送最终保存和停止事件，保证文档内容能够落盘。

```

77 class CollaborativeServer:

```

```

78     """ 多线程 TCP 服务端。
79
80     每个客户端连接会分配一个独立线程；所有线程共享同一个
81     CollaborativeDocument。文档内部自己持有锁，因此服务端这里主要负责
82     管理客户端连接表、把消息分发到文档模型，以及把结果广播给其他客户端。
83     """
84
85     def __init__(
86         self,
87         host: str,
88         port: int,
89         document_path: Path,
90         history_limit: int = 1000,
91         autosave_interval: float = 5.0,
92     ):
93         self.host = host
94         self.port = port
95         self.document_path = document_path
96         # multiprocessing 在 Windows 上可能重新导入模块并创建子进程。
97         # 记录创建服务端对象的进程号，可以避免子进程误执行父进程的 shutdown。
98         self._owner_pid = os.getpid()
99         # 如果目标文件已经存在，就把它作为协作文档初始内容；否则从空文档开始。
100         initial_text = document_path.read_text(encoding="utf-8") if
            ↳ document_path.exists() else ""
101         self.document = CollaborativeDocument(initial_text,
            ↳ history_limit=history_limit)
102         self.autosave_interval = autosave_interval
103         # clients 字典会被多个客户端线程同时读写，所以用 RLock 保护。
104         # 这个锁只保护“有哪些客户端在线”，不保护文档内容。
105         self.clients: Dict[str, ClientConnection] = {}
106         self.clients_lock = threading.RLock()
107         self._next_client_number = 1
108         self._stop_event = threading.Event()
109         self._server_socket: Optional[socket.socket] = None
110         # 自动保存放到独立进程中。主进程通过队列传递快照/日志事件，
111         # 避免在网络线程或文档锁临界区内执行磁盘 I/O。
112         self._autosave_queue: multiprocessing.Queue = multiprocessing.Queue()
113         self._autosave_process = multiprocessing.Process(
114             target=autosave_worker,
115             args=(
116                 self._autosave_queue,
117                 str(document_path),
118                 str(document_path.with_name(document_path.name + ".edit.log")),
119                 autosave_interval,
120             ),
121             daemon=True,
122         )

```

```
123
124 def serve_forever(self) -> None:
125     # 先启动保存进程，再把当前文档快照交给它。即使启动后暂时没有编辑，
126     # autosave 进程也知道当前版本和内容。
127     self._autosave_process.start()
128     self._autosave_queue.put(
129         {
130             "type": "SNAPSHOT",
131             "content": self.document.content(),
132             "version": self.document.version,
133         }
134     )
135     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_sock:
136         self._server_socket = server_sock
137         # SO_REUSEADDR 让服务端重启后能更快重新绑定同一个端口。
138         server_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
139         server_sock.bind((self.host, self.port))
140         server_sock.listen()
141         # accept 设置短超时，循环中才能定期检查 _stop_event。
142         server_sock.settimeout(0.5)
143         bound_host, bound_port = server_sock.getsockname()
144         print(f"Collaborative server listening on {bound_host}:{bound_port}",
145             ↪ flush=True)
146         print(f"Document: {self.document_path}", flush=True)
147         while not self._stop_event.is_set():
148             try:
149                 client_sock, address = server_sock.accept()
150             except socket.timeout:
151                 continue
152             except OSError:
153                 break
154             # TCP_NODELAY 禁用 Nagle 算法，减少按键级小消息的交互延迟。
155             client_sock.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
156             # 每个连接一个线程，适合课程中观察“多客户端并发连接”的模型。
157             thread = threading.Thread(
158                 target=self._handle_client,
159                 args=(client_sock, address),
160                 daemon=True,
161             )
162             thread.start()
163
164     self.shutdown()
165
166 def shutdown(self) -> None:
167     # 子进程里如果持有一份反序列化出来的 server 对象，也不能关闭父进程资源。
168     if os.getpid() != self._owner_pid:
```

```

169         return
170     # shutdown 可能由 Ctrl-C 信号、serve_forever 退出或测试重复触发,
171     # 因此需要做成幂等操作。
172     if self._stop_event.is_set():
173         return
174     self._stop_event.set()
175     if self._server_socket is not None:
176         try:
177             self._server_socket.close()
178         except OSError:
179             pass
180     self._autosave_queue.put(
181         {
182             "type": "SAVE_NOW",
183             "content": self.document.content(),
184             "version": self.document.version,

```

客户端消息处理：客户端线程先完成加入握手，并向新客户端返回欢迎消息和完整文档状态。后续消息由分发函数按类型处理：编辑操作进入文档模型，保存请求投递保存队列，文档请求则直接返回最新快照。操作处理完成后，服务端在文档锁外发送 ACK 和广播，从而避免慢客户端阻塞文档临界区。

```

193 def _handle_client(self, sock: socket.socket, address: Tuple[str, int]) -> None:
194     transport = JsonLineSocket(sock)
195     client: Optional[ClientConnection] = None
196     try:
197         # 协议规定客户端第一条消息必须是 JOIN。这样服务端可以先确定
198         # client_id, 再发送 WELCOME 和当前完整文档快照。
199         join_message = transport.recv_message()
200         if join_message.get("type") != "JOIN":
201             transport.send_message({"type": "ERROR", "message": "first message
202                                     ↳ must be JOIN"})
203             return
204         client_id = self._choose_client_id(join_message.get("client_id"))
205         client = ClientConnection(client_id=client_id, address=address,
206                                  ↳ transport=transport)
207         self._register_client(client)
208         client.send({"type": "WELCOME", "client_id": client_id})
209         # 新客户端必须先拿到完整 DOC_STATE, 之后才能基于具体版本提交 OP。
210         client.send(self.document.snapshot())
211         self._log({"event": "join", "client_id": client_id, "address":
212                   ↳ str(address)})
213
214     while not self._stop_event.is_set():
215         # 客户端线程只负责持续收消息并转交给分发函数;

```

```

213         # 具体业务处理集中在 _dispatch_message 中。
214         message = transport.recv_message()
215         self._dispatch_message(client, message)
216     except EOFError:
217         pass
218     except Exception as exc:
219         if client is not None:
220             self._safe_send(client, {"type": "ERROR", "message": str(exc)})
221     finally:
222         # 无论是正常断开、EOF 还是异常，都要把客户端从在线表移除。
223         if client is not None:
224             self._unregister_client(client.client_id)
225             self._log({"event": "leave", "client_id": client.client_id})
226         try:
227             sock.close()
228         except OSError:
229             pass
230
231     def _dispatch_message(self, client: ClientConnection, message: Message) -> None:
232         # 服务端协议是按 type 字段分发的。未知消息不会断开连接，
233         # 而是返回 ERROR，方便客户端或测试观察问题。
234         message_type = message.get("type")
235         if message_type == "OP":
236             self._handle_operation(client, message)
237         elif message_type == "SAVE":
238             # SAVE 不直接在客户端线程写文件，而是把当前快照投递给保存进程。
239             # 客户端收到 SAVE_QUEUED 表示请求已经进入保存队列。
240             snapshot = self.document.snapshot()
241             self._autosave_queue.put(
242                 {
243                     "type": "SAVE_NOW",
244                     "content": snapshot["content"],
245                     "version": snapshot["version"],
246                 }
247             )
248             self._print_event(
249                 format_save_event("requested", client.client_id, snapshot["version"],
250                                     ↪ self.document_path)
251             )
252             client.send({"type": "SAVE_QUEUED", "version": snapshot["version"]})
253         elif message_type == "REQUEST_DOC":
254             # 客户端发现版本断档或服务端要求重同步时，会主动请求完整文档。
255             client.send(self.document.snapshot())
256         elif message_type == "PING":
257             client.send({"type": "PONG"})
258         else:

```



```

258         client.send({"type": "ERROR", "message": f"unsupported message type:
    ↪ {message_type!r}"})
259
260 def _handle_operation(self, client: ClientConnection, message: Message) -> None:
261     # process_operation 内部会获取文档锁，并在临界区内完成 OT、
262     # server_version 分配和文档更新。函数返回后，文档锁已经释放。
263     result = self.document.process_operation(
264         client_id=client.client_id,
265         op_id=str(message.get("op_id", "")),
266         base_version=message.get("base_version", -1),
267         op_payload=message.get("op", {}),
268     )
269
270     if result.status == "ok":
271         self._record_successful_operation(result)
272         # 注意: ACK 和 REMOTE_OP 都在文档锁外发送。
273         # 这样即使某个客户端网络很慢，也不会阻塞其他线程进入文档临界区。
274         if result.ack is not None:
275             client.send(result.ack)
276         if result.remote is not None:
277             self._broadcast(result.remote, exclude_client_id=client.client_id)
278         return
279
280     for response in result.messages:
281         client.send(response)
282

```

广播、日志与命令行入口：服务端将成功操作的快照交给自动保存进程，把结构化事件写入日志，并在复制客户端列表后执行广播。最后的入口函数通过 `argparse` 提供监听地址、端口、协作文档路径、历史长度和自动保存间隔等参数，并注册信号处理函数，使 `Ctrl-C` 停止服务端时也能走统一清理流程。

```

283 def _record_successful_operation(self, result: ProcessResult) -> None:
284     # 文档模型返回的 ProcessResult 已经把“要保存的快照”和“要写的日志”
285     # 与网络 ACK/广播消息分开，服务端在这里分别投递到对应通道。
286     if result.snapshot_content is not None and result.snapshot_version is not
    ↪ None:
287         self._autosave_queue.put(
288             {
289                 "type": "SNAPSHOT",
290                 "content": result.snapshot_content,
291                 "version": result.snapshot_version,
292             }
293         )
294     if result.log_event is not None:

```

```

295         self._log(result.log_event)
296
297     def _choose_client_id(self, requested: object) -> str:
298         # 如果客户端请求的 id 可用就保留, 方便演示时指定 u1/u2。
299         # 冲突或空 id 则由服务端分配递增编号。
300         with self.clients_lock:
301             if isinstance(requested, str) and requested and requested not in
302                 ↪ self.clients:
303                 return requested
304             while True:
305                 candidate = f"u{self._next_client_number}"
306                 self._next_client_number += 1
307                 if candidate not in self.clients:
308                     return candidate
309
310     def _register_client(self, client: ClientConnection) -> None:
311         # 在线客户端表是广播的收件人来源, 必须在锁内更新。
312         with self.clients_lock:
313             self.clients[client.client_id] = client
314
315     def _unregister_client(self, client_id: str) -> None:
316         # 客户端可能已经因为发送失败被提前移除, 所以 pop 使用默认值保持幂等。
317         with self.clients_lock:
318             self.clients.pop(client_id, None)
319
320     def _broadcast(self, message: Message, exclude_client_id: Optional[str] = None)
321         ↪ -> None:
322         # 持有客户端注册表锁时只复制收件人列表, 释放锁之后再执行套接字 I/O。
323         # 这里的锁只保护 clients 字典, 不保护文档内容。复制收件人列表后立刻释放,
324         # 可以避免广播阶段和客户端加入/退出管理互相长时间阻塞。
325         with self.clients_lock:
326             recipients: List[ClientConnection] = [
327                 client
328                 for cid, client in self.clients.items()
329                 if cid != exclude_client_id
330             ]
331             failed: List[str] = []
332             for recipient in recipients:
333                 if not self._safe_send(recipient, message):
334                     failed.append(recipient.client_id)
335             for client_id in failed:
336                 self._unregister_client(client_id)
337
338     def _safe_send(self, client: ClientConnection, message: Message) -> bool:
339         # 广播时某个客户端断开不应该影响其他客户端, 因此发送失败只返回 False。
340         try:
341             client.send(message)

```

```
340         return True
341     except OSError:
342         return False
343
344     def _log(self, entry: Message) -> None:
345         # 同一份结构化事件既进入磁盘日志，也转换为终端输出。
346         self._autosave_queue.put({"type": "LOG", "entry": entry})
347         self._print_event(format_server_event(entry))
348
349     def _print_event(self, message: Optional[str]) -> None:
350         if message:
351             print(message, flush=True)
352
353
354     def parse_args() -> argparse.Namespace:
355         # 命令行参数保持简单，便于在 Linux/WSL 终端中直接启动服务端演示。
356         parser = argparse.ArgumentParser(description=" 服务端权威 OT 协作编辑器")
357         parser.add_argument("--host", default="127.0.0.1")
358         parser.add_argument("--port", type=int, default=8765)
359         parser.add_argument("--file", default="shared.py", help=" 要协作编辑的文档路径")
360         parser.add_argument("--history", type=int, default=1000)
361         parser.add_argument("--autosave-interval", type=float, default=5.0)
362         return parser.parse_args()
363
364
365     def main() -> None:
366         args = parse_args()
367         server = CollaborativeServer(
368             host=args.host,
369             port=args.port,
370             document_path=Path(args.file),
371             history_limit=args.history,
372             autosave_interval=args.autosave_interval,
373         )
374
375         def stop(_signum: int, _frame: object) -> None:
376             # Ctrl-C / SIGTERM 触发正常收尾：最终保存、通知 autosave 进程退出。
377             server.shutdown()
378
379         signal.signal(signal.SIGINT, stop)
380         signal.signal(signal.SIGTERM, stop)
381         server.serve_forever()
382
383
384     if __name__ == "__main__":
385         main()
```

8.4 客户端状态机与终端界面

客户端同步状态：客户端同步状态的关键是 `pending_queue` 和 `inflight_op`。用户连续输入产生的操作先进入队列；同一时刻客户端最多发送一个未确认操作，收到服务端 ACK 后再发送下一个。客户端应用的始终是服务端返回的权威操作，因此不需要在本地实现复杂 OT。

```

1  """ 客户端待发送队列和权威状态应用逻辑。
2
3  客户端并不尝试自己解决并发冲突。它只负责收集本地按键、按顺序发送，
4  并在收到服务端 ACK/REMOTE_OP 后把权威操作应用到本地文本。
5  """
6
7  from __future__ import annotations
8
9  from collections import deque
10 from dataclasses import dataclass
11 from typing import Callable, Deque, Dict, Optional
12
13 from document import Operation, apply_operation_to_text
14 from protocol import Message
15
16
17 @dataclass
18 class PendingOperation:
19     """ 客户端本地尚未完全确认的一条操作。
20
21     base_version 在真正发出时才填写，因为排队期间客户端版本可能会变化。
22     """
23
24     op_id: str
25     op: Operation
26     base_version: Optional[int] = None
27
28
29 class ClientSyncState:
30     """ 缓存本地输入，并保证同一时刻最多只有一个未确认操作。 """
31
32     def __init__(self, client_id: str, send_message: Callable[[Message], None]):
33         self.client_id = client_id
34         # send_message 由外层网络层注入，便于单元测试用 list.append 代替 socket。
35         self.send_message = send_message
36         # content/version 保存客户端当前已经确认的权威状态。
37         self.content = ""
38         self.version = 0

```

```

39     self.history_start_version = 0
40     # pending_queue 保存用户已经输入、但还没有轮到发送的操作。
41     self.pending_queue: Deque[PendingOperation] = deque()
42     # inflight_op 表示已经发送给服务端、正在等待 ACK 的唯一操作。
43     self.inflight_op: Optional[PendingOperation] = None
44     self._next_op_number = 1
45     self.status = "disconnected"
46
47     def set_document(self, content: str, version: int, history_start_version: int =
↪ 0) -> None:
48         # 完整文档状态来自服务端 DOC_STATE。收到它时，客户端以服务端为准，
49         # 清空当前 inflight，重新从这个版本继续发送本地排队操作。
50         self.content = content
51         self.version = version
52         self.history_start_version = history_start_version
53         self.inflight_op = None
54         self.status = f"synced at version {version}"
55         self._pump()
56
57     def queue_operation(self, op: Operation) -> str:
58         op_id = f"{self.client_id}-{self._next_op_number}"
59         self._next_op_number += 1
60         # 用户连续按键时不能因为上一个操作还没 ACK 就丢输入。
61         # 所有本地输入先进入 pending_queue，再由 _pump 按顺序发送。
62         self.pending_queue.append(PendingOperation(op_id=op_id, op=op))
63         self.status = f"queued {len(self.pending_queue)} operation(s)"
64         self._pump()
65         return op_id
66
67     def handle_message(self, message: Message) -> None:
68         # 客户端收到的所有服务端消息都在这里转换为本地状态变化。
69         message_type = message.get("type")
70         if message_type == "DOC_STATE":
71             self.set_document(
72                 content=str(message.get("content", "")),
73                 version=int(message.get("version", 0)),
74                 history_start_version=int(message.get("history_start_version", 0)),
75             )
76         elif message_type == "WELCOME":
77             # 如果请求的 client_id 冲突，服务端会分配新 id；之后 op_id 也用新 id。
78             self.client_id = str(message.get("client_id", self.client_id))
79             self.status = f"connected as {self.client_id}"
80         elif message_type == "ACK":
81             self._handle_ack(message)
82         elif message_type == "REMOTE_OP":
83             self._handle_remote_op(message)
84         elif message_type == "RESYNC_REQUIRED":

```

```

85         # 服务端历史不足以变换当前操作时, 客户端主动请求完整文档。
86         # inflight 被丢弃, 因为它的基准版本已经不能安全使用。
87         self.status = "server requested resync"
88         self.inflight_op = None
89         self.send_message({"type": "REQUEST_DOC"})
90     elif message_type == "SAVE_QUEUED":
91         self.status = f"save queued at version {message.get('version')}"
92     elif message_type == "ERROR":
93         self.status = f"error: {message.get('message')}"
94
95     def _handle_ack(self, message: Message) -> None:
96         op_id = str(message.get("op_id", ""))
97         # 网络延迟或重连场景下可能收到过期 ACK; 只接受当前 inflight 的 ACK。
98         if self.inflight_op is None or self.inflight_op.op_id != op_id:
99             self.status = f"ignored stale ACK {op_id}"
100             return
101         # 客户端不自行决定本地操作最终落点, 而是等待服务端 ACK 中的
102         # 权威 op。这个 op 可能已经被服务端 OT 变换过。
103         self._apply_authoritative_op(message)
104         self.inflight_op = None
105         self.status = f"ACK {op_id} at version {self.version}"
106         self._pump()
107
108     def _handle_remote_op(self, message: Message) -> None:
109         incoming_version = int(message.get("version", 0))
110         # 已经见过的远端操作可以忽略, 防止重复应用。
111         if incoming_version <= self.version:
112             return
113         if incoming_version != self.version + 1:
114             # 如果远端版本不是紧接着当前版本, 说明中间漏了一些操作。
115             # 客户端不猜测缺口, 直接请求完整文档恢复一致。
116             self.status = "missed remote operation; requesting document"
117             self.send_message({"type": "REQUEST_DOC"})
118             return
119         self._apply_authoritative_op(message)
120         self.status = f"remote op at version {self.version}"
121
122     def _apply_authoritative_op(self, message: Message) -> None:
123         # 本地文本永远应用服务端给出的权威 op, 而不是自己原先排队的 op。
124         # 这样客户端不需要实现 OT, 也能跟随服务端最终顺序。
125         op = Operation.from_dict(message.get("op", {"kind": "noop"}))
126         self.content = apply_operation_to_text(self.content, op)
127         self.version = int(message.get("version", self.version))
128
129     def _pump(self) -> None:
130         if self.inflight_op is not None or not self.pending_queue:
131             return

```

```

132     next_op = self.pending_queue.popleft()
133     next_op.base_version = self.version
134     self.inflight_op = next_op
135     # 每个客户端同一时刻只发送一个未确认操作。
136     # 这样客户端侧不需要实现复杂 OT, 只要记住当前版本并等待 ACK。
137     self.send_message(
138         {
139             "type": "OP",
140             "client_id": self.client_id,
141             "op_id": next_op.op_id,
142             "base_version": next_op.base_version,
143             "op": next_op.op.to_dict(),
144         }
145     )

```

终端宽字符与光标换算：终端客户端需要在一维文档索引、行列坐标和实际屏幕显示列之间转换。由于中文等宽字符在终端中通常占两个显示列,代码中使用 `unicodedata` 判断字符宽度,并提供显示列与字符列之间的转换函数,保证中英文混排时光标移动仍能对齐。

```

1  """ 协作编辑器的 curses 终端客户端。
2
3  客户端负责把用户在终端中的按键转换为字符级 Operation, 并把服务端返回的
4  权威文档状态渲染出来。它不做复杂冲突解决,冲突处理统一交给服务端 OT。
5  """
6
7  from __future__ import annotations
8
9  import argparse
10 import locale
11 import queue
12 import socket
13 import threading
14 import unicodedata
15 from typing import List, Tuple, Union
16
17 try:
18     import curses
19 except ModuleNotFoundError:
20     # Windows 原生 Python 通常没有 curses; WSL/Linux 终端中可以正常运行。
21     curses = None # type: ignore[assignment]
22
23 from client_state import ClientSyncState
24 from document import Operation
25 from protocol import JsonLineSocket, Message

```



```
26
27
28 CTRL_Q = 17
29 CTRL_S = 19
30 CONTROL_Q = "\x11"
31 CONTROL_S = "\x13"
32 # 不同终端对 Backspace 的编码可能不同, 因此同时接受常见形式。
33 BACKSPACE_KEYS = {"\b", "\x7f"}
34 # 这些控制字符在本编辑器里没有定义行为, 避免误当作普通字符插入。
35 IGNORED_INPUT = {"\t", "\r", "\x0b", "\x0c"}
36 KeyInput = Union[int, str]
37
38
39 def pos_to_line_col(content: str, pos: int) -> Tuple[int, int]:
40     # 文档内部用“从头开始的字符索引”表示光标位置;
41     # curses 渲染时需要转换成第几行、第几列。
42     pos = max(0, min(pos, len(content)))
43     line = content.count("\n", 0, pos)
44     last_newline = content.rfind("\n", 0, pos)
45     col = pos if last_newline < 0 else pos - last_newline - 1
46     return line, col
47
48
49 def char_display_width(char: str) -> int:
50     # len("你") 是 1, 但在大多数终端里占 2 个显示列。
51     # curses move 需要的是屏幕列, 因此要单独计算显示宽度。
52     if not char:
53         return 0
54     if unicodedata.combining(char):
55         return 0
56     if unicodedata.east_asian_width(char) in {"F", "W"}:
57         return 2
58     return 1
59
60
61 def text_display_width(text: str) -> int:
62     # 逐字符累加显示宽度, 支持中英文混排时的光标定位。
63     return sum(char_display_width(char) for char in text)
64
65
66 def display_col_to_char_col(text: str, display_col: int) -> int:
67     # 把屏幕显示列转换回字符串字符索引。
68     # 向上/向下移动时, 希望尽量保持“视觉列”不变, 而不是简单保持字符数。
69     display_col = max(0, display_col)
70     current_col = 0
71     for index, char in enumerate(text):
72         width = char_display_width(char)
```

```

73         if current_col + width > display_col:
74             return index
75         current_col += width
76     return len(text)
77
78
79 def line_col_to_pos_by_display(content: str, line: int, display_col: int) -> int:
80     # 根据目标行和视觉列找到文档中的一维字符索引。
81     # 主要用于上下方向键处理宽字符时的光标位置。
82     lines = content.splitlines(keepends=True)
83     if not lines:
84         return 0
85     line = max(0, min(line, len(lines) - 1))
86     line_start = sum(len(part) for part in lines[:line])
87     line_text = lines[line].rstrip("\n")
88     return line_start + display_col_to_char_col(line_text, display_col)
89
90
91 def line_col_to_pos(content: str, line: int, col: int) -> int:
92     # 根据普通字符列计算一维索引。当前代码主要保留作通用工具，
93     # 与 display 版本相比，它不考虑中文等宽字符的屏幕宽度。
94     lines = content.splitlines(keepends=True)
95     if not lines:
96         return 0
97     line = max(0, min(line, len(lines) - 1))
98     line_start = sum(len(part) for part in lines[:line])
99     line_text = lines[line]
100     visible_len = len(line_text.rstrip("\n"))
101     return line_start + max(0, min(col, visible_len))
102
103
104 def split_lines_for_display(content: str) -> List[str]:
105     # str.split("\n") 会在空文档时返回 [""], 正好可以渲染出一行空行。
106     lines = content.split("\n")
107     return lines if lines else [""]
108
109
110 def is_insertable_character(value: str) -> bool:
111     # 只允许单个、非控制字符进入文档；快捷键和终端控制字符在 _handle_key 里处理。
112     if len(value) != 1 or value in IGNORED_INPUT:
113         return False
114     return unicodedata.category(value)[0] != "C"

```

curses 客户端主体: `TerminalClient` 把网络接收和 curses 界面分开: 后台线程只负责从 socket 读取消息并放入队列, 界面线程负责消费消息、渲染状态栏和正文, 并把用户按键转换为 `Operation`。普通字符、回车和 Backspace 会变成协作编辑操作,

Ctrl-S 和 Ctrl-Q 分别用于保存和退出。

```

117 class TerminalClient:
118     """curses 终端客户端。
119
120     网络读取在后台线程中完成，UI 主循环只从 incoming 队列取消息并渲染。
121     这样服务端消息到达时不会卡住键盘输入，也不会让 curses 调用跨线程执行。
122     """
123
124     def __init__(self, host: str, port: int, client_id: str):
125         self.host = host
126         self.port = port
127         self.client_id = client_id
128         self.transport: JsonLineSocket
129         self.state: ClientSyncState
130         # reader_thread 把 socket 消息放进队列，UI 线程再统一消费。
131         self.incoming: "queue.Queue[Message]" = queue.Queue()
132         self.reader_thread: threading.Thread
133         # cursor_pos 是文档中的字符索引，不是屏幕坐标。
134         self.cursor_pos = 0
135         # scroll_line 表示当前屏幕正文区域顶部对应文档的哪一行。
136         self.scroll_line = 0
137         self.running = True
138
139     def connect(self) -> None:
140         # TCP_NODELAY 降低按键消息延迟；协作编辑通常每次只发送很小的 JSON。
141         sock = socket.create_connection((self.host, self.port))
142         sock.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
143         self.transport = JsonLineSocket(sock)
144         self.state = ClientSyncState(self.client_id, self.transport.send_message)
145         # JOIN 是协议握手第一步，服务端随后返回 WELCOME 和 DOC_STATE。
146         self.transport.send_message({"type": "JOIN", "client_id": self.client_id})
147         self.reader_thread = threading.Thread(target=self._reader_loop, daemon=True)
148         self.reader_thread.start()
149
150     def run(self) -> None:
151         if curses is None:
152             raise RuntimeError("client.py requires curses; please run it in Linux or
153                 ↪ WSL")
154         self.connect()
155         # curses.wrapper 会初始化/恢复终端状态，即使异常退出也尽量把终端还原。
156         curses.wrapper(self._curses_main)
157
158     def _reader_loop(self) -> None:
159         # 后台线程只做阻塞 recv，不直接碰 curses 界面。
160         while self.running:
161             try:

```

```

161         self.incoming.put(self.transport.recv_message())
162     except Exception as exc:
163         self.incoming.put({"type": "ERROR", "message": str(exc)})
164     return
165
166 def _curses_main(self, stdscr: "curses._CursesWindow") -> None:
167     curses.curs_set(1)
168     # nodelay 让 get_wch 没有按键时立即抛 curses.error,
169     # 主循环就能持续刷新远端更新, 而不是一直阻塞等键盘输入。
170     stdscr.nodelay(True)
171     stdscr.keypad(True)
172     while self.running:
173         self._drain_messages()
174         self._render(stdscr)
175         try:
176             key = stdscr.get_wch()
177         except curses.error:
178             # 没有按键时短暂休眠, 避免空转占满 CPU。
179             curses.napms(25)
180             continue
181         self._handle_key(key)
182
183 def _drain_messages(self) -> None:
184     # 一次性取空队列, 保证渲染前本地状态尽量追上服务端。
185     while True:
186         try:
187             message = self.incoming.get_nowait()
188         except queue.Empty:
189             return
190         old_content = self.state.content
191         self.state.handle_message(message)
192         if self.state.content != old_content:
193             # 远端操作可能改变文档长度, 光标要夹在合法范围内。
194             self.cursor_pos = max(0, min(self.cursor_pos,
195                                         ↪ len(self.state.content)))
196
197 def _render(self, stdscr: "curses._CursesWindow") -> None:
198     stdscr.erase()
199     height, width = stdscr.getmaxyx()
200     # 顶部状态栏展示客户端 id、服务端版本、待发送队列和当前同步状态。
201     status = (
202         f"{self.client_id} | v{self.state.version} | "
203         f"pending={len(self.state.pending_queue)} | {self.state.status}"
204     )
205     stdscr.addnstr(0, 0, status, max(0, width - 1), curses.A_REVERSE)
206
207     lines = split_lines_for_display(self.state.content)

```

```

207     cursor_line, cursor_col = pos_to_line_col(self.state.content,
        ↪ self.cursor_pos)
208     # 根据光标所在行调整滚动窗口，保证光标尽量保持在可视区域。
209     if cursor_line < self.scroll_line:
210         self.scroll_line = cursor_line
211     if cursor_line >= self.scroll_line + height - 2:
212         self.scroll_line = cursor_line - height + 3
213
214     for row in range(1, height - 1):
215         line_index = self.scroll_line + row - 1
216         if line_index >= len(lines):
217             break
218         # 左侧固定显示行号，正文从第 6 列开始。
219         prefix = f"{line_index + 1:>4} "
220         body_width = max(0, width - len(prefix) - 1)
221         stdscr.addnstr(row, 0, prefix, width - 1, curses.A_DIM)
222         stdscr.addnstr(row, len(prefix), lines[line_index], body_width)
223
224     help_text = "Ctrl-S save | Ctrl-Q quit"
225     stdscr.addnstr(height - 1, 0, help_text, max(0, width - 1), curses.A_REVERSE)
226     screen_row = cursor_line - self.scroll_line + 1
227     current_line = lines[cursor_line] if cursor_line < len(lines) else ""
228     # cursor_col 是字符列，screen_col 需要按实际显示宽度换算。
229     screen_col = text_display_width(current_line[:cursor_col]) + 5
230     if 1 <= screen_row < height - 1 and 0 <= screen_col < width:
231         stdscr.move(screen_row, screen_col)
232     stdscr.refresh()
233
234     def _handle_key(self, key: KeyInput) -> None:
235         # curses 可能把 Ctrl 键返回为整数，也可能返回控制字符串，
236         # 所以这里同时兼容两种形式。
237         if key in (CTRL_Q, CONTROL_Q):
238             self.running = False
239             return
240         if key in (CTRL_S, CONTROL_S):
241             self.transport.send_message({"type": "SAVE"})
242             return
243         if key in BACKSPACE_KEYS:
244             self._delete_before_cursor()
245             return
246         if isinstance(key, str):
247             # 普通字符输入先生成本地 Operation，由 ClientSyncState 排队发送。
248             if key == "\n":
249                 self.state.queue_operation(Operation("insert", self.cursor_pos,
                ↪ "\n"))
250                 self.cursor_pos += 1
251             return

```

```

252         if is_insertable_character(key):
253             self.state.queue_operation(Operation("insert", self.cursor_pos, key))
254             self.cursor_pos += 1
255         return
256     if key == curses.KEY_LEFT:
257         # 左右移动只改变本地光标，不产生协作操作。
258         self.cursor_pos = max(0, self.cursor_pos - 1)
259         return
260     if key == curses.KEY_RIGHT:
261         self.cursor_pos = min(len(self.state.content), self.cursor_pos + 1)
262         return
263     if key == curses.KEY_UP:
264         line, col = pos_to_line_col(self.state.content, self.cursor_pos)
265         current_line = split_lines_for_display(self.state.content)[line]
266         # 上下移动保持视觉列，避免光标穿过中文宽字符时左右跳动。
267         display_col = text_display_width(current_line[:col])
268         self.cursor_pos = line_col_to_pos_by_display(self.state.content, line -
269             ↪ 1, display_col)
270         return
271     if key == curses.KEY_DOWN:
272         line, col = pos_to_line_col(self.state.content, self.cursor_pos)
273         current_line = split_lines_for_display(self.state.content)[line]
274         display_col = text_display_width(current_line[:col])
275         self.cursor_pos = line_col_to_pos_by_display(self.state.content, line +
276             ↪ 1, display_col)
277         return
278     if key in (curses.KEY_BACKSPACE, 127, 8):
279         self._delete_before_cursor()
280         return
281     if key in (curses.KEY_ENTER, 10, 13):
282         self.state.queue_operation(Operation("insert", self.cursor_pos, "\n"))
283         self.cursor_pos += 1
284
285     def _delete_before_cursor(self) -> None:
286         # Backspace 删除光标前一个字符，并把光标移动到被删字符的位置。
287         if self.cursor_pos > 0:
288             delete_pos = self.cursor_pos - 1
289             self.state.queue_operation(Operation("delete", delete_pos))
290             self.cursor_pos = delete_pos

```

客户端入口：客户端入口读取服务端地址、端口和客户端编号，并设置系统 locale，使 curses 能够在终端中正确处理中文输入和宽字符显示。

```

291 def parse_args() -> argparse.Namespace:
292     parser = argparse.ArgumentParser(description=" 终端协作编辑器客户端")

```

```

293     parser.add_argument("--host", default="127.0.0.1")
294     parser.add_argument("--port", type=int, default=8765)
295     parser.add_argument("--client-id", default="u1")
296     return parser.parse_args()
297
298
299 def main() -> None:
300     locale.setlocale(locale.LC_ALL, "")
301     args = parse_args()
302     client = TerminalClient(args.host, args.port, args.client_id)
303     client.run()
304
305
306 if __name__ == "__main__":
307     main()

```

8.5 自动保存、备份与日志

保存进程：自动保存和日志写入放在独立进程中完成，主服务端只通过进程间队列投递快照、保存请求和日志事件。`atomic_write_text` 使用临时文件替换目标文件，降低写入中断造成半截文件的风险；手动保存会先复制旧文件生成 `.bak`，再写入正式文件；自动保存则写入独立的 `.autosave` 文件。

```

1  """ 自动保存和编辑日志工作进程。
2
3  服务端把磁盘写入工作交给这个模块中的 worker 进程完成。这样主服务端线程
4  处理网络请求和文档锁时，不会因为文件系统 I/O 变慢而卡住其他客户端。
5  """
6
7  from __future__ import annotations
8
9  import json
10 import signal
11 import shutil
12 import time
13 from multiprocessing import Queue
14 from pathlib import Path
15 from queue import Empty
16 from typing import Any, Dict, Optional
17
18
19 def atomic_write_text(path: Path, content: str) -> None:
20     # 先写到临时文件，再用 replace 一次性替换目标文件。
21     # 这样即使程序在写入中途崩溃，也尽量避免留下半截正式文件。
22     path.parent.mkdir(parents=True, exist_ok=True)

```



```
23     tmp_path = path.with_name(path.name + ".tmp")
24     tmp_path.write_text(content, encoding="utf-8")
25     tmp_path.replace(path)
26
27
28 def write_with_backup(path: Path, content: str) -> None:
29     # 手动保存会覆盖正式文件，所以覆盖前先复制一份 .bak。
30     # 自动保存只写 .autosave，不走这里。
31     path.parent.mkdir(parents=True, exist_ok=True)
32     if path.exists():
33         backup_path = path.with_name(path.name + ".bak")
34         shutil.copy2(path, backup_path)
35     atomic_write_text(path, content)
36
37
38 def append_json_log(path: Path, entry: Dict[str, Any]) -> None:
39     # 日志使用 JSON Lines: 每一行是一个独立 JSON 对象，便于 tail 查看，
40     # 也便于后续用脚本逐行分析。
41     path.parent.mkdir(parents=True, exist_ok=True)
42     with path.open("a", encoding="utf-8") as handle:
43         handle.write(json.dumps(entry, ensure_ascii=False, sort_keys=True) + "\n")
44
45
46 def autosave_worker(
47     event_queue: Queue,
48     document_path: str,
49     log_path: str,
50     interval_seconds: float = 5.0,
51 ) -> None:
52     # 保存进程不响应 Ctrl-C，由父进程负责发送 STOP 做有序退出。
53     signal.signal(signal.SIGINT, signal.SIG_IGN)
54     # 自动保存和日志写入放在单独进程中完成。
55     # 主服务端只需要通过 multiprocessing.Queue 投递事件，不必在处理
56     # 网络请求或文档锁时直接做磁盘 I/O，从而体现进程间通信的使用。
57     target_path = Path(document_path)
58     autosave_path = target_path.with_name(target_path.name + ".autosave")
59     log_file = Path(log_path)
60     latest_content: Optional[str] = None
61     latest_version = 0
62     # 使用 monotonic 计算间隔，避免系统时间被手动调整影响自动保存节奏。
63     next_save = time.monotonic() + interval_seconds
64
65     while True:
66         # 队列 get 的超时时间正好是“距离下一次自动保存还剩多久”。
67         # 如果超时没有新事件，就执行一次定时保存检查。
68         timeout = max(0.0, next_save - time.monotonic())
69         try:
```

```
70     event = event_queue.get(timeout=timeout)
71 except Empty:
72     event = None
73
74 if event:
75     event_type = event.get("type")
76     if event_type == "STOP":
77         # 退出前把最后一份快照写入 autosave 文件，减少关闭时的数据丢失窗口。
78         if latest_content is not None:
79             atomic_write_text(autosave_path, latest_content)
80         return
81     if event_type == "SNAPSHOT":
82         # 服务端每处理完一次成功编辑，就发送最新快照。
83         # 工作进程只保存最后一次快照，定时落盘，避免每个按键都写文件。
84         latest_content = event.get("content", "")
85         latest_version = int(event.get("version", latest_version))
86     elif event_type == "SAVE_NOW":
87         # 手动保存写入正式文件，并在覆盖前生成 .bak 备份。
88         content = event.get("content", latest_content or "")
89         latest_content = content
90         latest_version = int(event.get("version", latest_version))
91         write_with_backup(target_path, content)
92         append_json_log(
93             log_file,
94             {
95                 "event": "manual_save",
96                 "version": latest_version,
97                 "path": str(target_path),
98                 "timestamp": time.time(),
99             },
100         )
101     elif event_type == "LOG":
102         # 普通编辑日志不改文档文件，只追加一行 JSON。
103         entry = dict(event.get("entry", {}))
104         entry.setdefault("timestamp", time.time())
105         append_json_log(log_file, entry)
106
107 if time.monotonic() >= next_save:
108     if latest_content is not None:
109         # 自动保存写的是旁路快照文件，不覆盖正式文件。
110         atomic_write_text(autosave_path, latest_content)
111         append_json_log(
112             log_file,
113             {
114                 "event": "autosave",
115                 "version": latest_version,
116                 "path": str(autosave_path),
```

```

117         "timestamp": time.time(),
118     },
119 )
120 next_save = time.monotonic() + interval_seconds

```

8.6 单元测试

文档一致性测试：这一组测试直接验证服务端权威文档模型，也是 OT 正确性的核心证据。测试包括旧版本删除位置经过多次插入后右移、重复删除同一字符变为 noop、历史过期触发重同步、服务端版本严格递增，以及中文字符能够作为单字符插入。

```

1 import unittest
2
3 from document import CollaborativeDocument
4
5
6 class CollaborativeDocumentTests(unittest.TestCase):
7     """ 验证服务端权威文档模型的 OT、版本和历史裁剪行为。 """
8
9     def test_delete_position_transforms_after_prior_inserts(self):
10         # 场景: A 已经在旧位置附近连续插入, B 仍基于版本 0 删除原来的空格。
11         # 服务端必须把 B 的删除位置向右平移, 才能删除“原本想删的那个字符”。
12         doc = CollaborativeDocument("hello world")
13         for index, pos in enumerate([5, 6, 7, 8], start=1):
14             result = doc.process_operation(
15                 client_id="A",
16                 op_id=f"A-{index}",
17                 base_version=index - 1,
18                 op_payload={"kind": "insert", "pos": pos, "char": "!"},
19             )
20             self.assertEqual(result.status, "ok")
21
22         result = doc.process_operation(
23             client_id="B",
24             op_id="B-1",
25             base_version=0,
26             op_payload={"kind": "delete", "pos": 5},
27         )
28
29         self.assertEqual(result.status, "ok")
30         # 四次插入之后, B 的删除成为第 5 个服务端权威操作。
31         self.assertEqual(result.ack["server_version"], 5)
32         # 原始 pos=5 经过四次 insert 变换后落在 pos=9。
33         self.assertEqual(result.ack["op"], {"kind": "delete", "pos": 9})
34         self.assertEqual(doc.content(), "hello!!!!world")

```

```
35
36 def test_same_character_delete_becomes_noop(self):
37     # 两个客户端都基于版本 0 删除同一个字符。第一个删除生效后,
38     # 第二个删除不能再删一次, 否则会误删后面的字符, 所以应变成 noop。
39     doc = CollaborativeDocument("abc")
40     first = doc.process_operation(
41         client_id="A",
42         op_id="A-1",
43         base_version=0,
44         op_payload={"kind": "delete", "pos": 1},
45     )
46     second = doc.process_operation(
47         client_id="B",
48         op_id="B-1",
49         base_version=0,
50         op_payload={"kind": "delete", "pos": 1},
51     )
52
53     self.assertEqual(first.status, "ok")
54     self.assertEqual(second.status, "ok")
55     # noop 仍然会获得 ACK 和版本号, 保证所有客户端版本推进一致。
56     self.assertEqual(second.ack["op"]["kind"], "noop")
57     self.assertEqual(doc.content(), "ac")
58     self.assertEqual(doc.version, 2)
59
60 def test_history_expiry_requests_resync(self):
61     # history_limit=2 表示服务端只保留最近两条操作。
62     # 当客户端基于更旧版本提交操作时, 服务端已经无法安全做 OT。
63     doc = CollaborativeDocument("", history_limit=2)
64     for version in range(3):
65         result = doc.process_operation(
66             client_id="A",
67             op_id=f"A-{version}",
68             base_version=version,
69             op_payload={"kind": "insert", "pos": version, "char": "x"},
70         )
71         self.assertEqual(result.status, "ok")
72
73     self.assertEqual(doc.history_start_version, 1)
74     expired = doc.process_operation(
75         client_id="B",
76         op_id="B-1",
77         base_version=0,
78         op_payload={"kind": "insert", "pos": 0, "char": "y"},
79     )
80
81     self.assertEqual(expired.status, "resync")
```

```

82     # resync 响应同时包含原因消息和完整文档快照，客户端可立即恢复状态。
83     self.assertEqual(expired.messages[0]["type"], "RESYNC_REQUIRED")
84     self.assertEqual(expired.messages[1]["type"], "DOC_STATE")
85
86     def test_server_version_is_strictly_incremented(self):
87         # 每个被接受的操作都必须拿到严格递增的 server_version,
88         # 这是所有客户端判断全局顺序的依据。
89         doc = CollaborativeDocument("")
90         versions = []
91         for index, char in enumerate("abc"):
92             result = doc.process_operation(
93                 client_id="A",
94                 op_id=f"A-{index}",
95                 base_version=index,
96                 op_payload={"kind": "insert", "pos": index, "char": char},
97             )
98             versions.append(result.ack["server_version"])
99
100         self.assertEqual(versions, [1, 2, 3])
101         self.assertEqual(doc.content(), "abc")
102
103     def test_insert_chinese_character(self):
104         # Python 字符串按 Unicode 字符计数，中文“你”应被视为单个插入字符。
105         doc = CollaborativeDocument("")
106         result = doc.process_operation(
107             client_id="A",
108             op_id="A-1",
109             base_version=0,
110             op_payload={"kind": "insert", "pos": 0, "char": "你"},
111         )
112
113         self.assertEqual(result.status, "ok")
114         self.assertEqual(result.ack["server_version"], 1)
115         self.assertEqual(doc.content(), "你")
116
117
118 if __name__ == "__main__":
119     unittest.main()
120

```

客户端状态测试：这一组测试说明客户端为什么不会在连续输入时丢操作。它验证了 ACK 驱动发送队列、远程版本缺口请求完整文档、服务端重新分配客户端编号、中文字符可插入、宽字符显示宽度计算，以及 Backspace 会排队生成删除操作。

```

1 import unittest
2

```

```

3 from client import BACKSPACE_KEYS, TerminalClient, display_col_to_char_col,
  ↪ is_insertable_character, text_display_width
4 from client_state import ClientSyncState
5 from document import Operation
6
7
8 class ClientSyncStateTests(unittest.TestCase):
9     """ 验证客户端状态机的发送节奏、重同步和宽字符辅助逻辑。 """
10
11     def test_pending_queue_sends_next_only_after_ack(self):
12         # 客户端连续输入两个字符时，第二个操作先留在 pending_queue。
13         # 只有第一个操作收到 ACK 后，客户端才用新版本发送第二个操作。
14
15         sent = []
16         state = ClientSyncState("u1", sent.append)
17         state.set_document("abc", 0)
18
19         first_id = state.queue_operation(Operation("insert", 3, "x"))
20         second_id = state.queue_operation(Operation("insert", 4, "y"))
21
22         self.assertEqual(len(sent), 1)
23         # 第一个操作基于初始版本 0 发出；第二个还没有进入网络层。
24         self.assertEqual(sent[0]["op_id"], first_id)
25         self.assertEqual(sent[0]["base_version"], 0)
26         self.assertEqual(state.inflight_op.op_id, first_id)
27         self.assertEqual(len(state.pending_queue), 1)
28
29         state.handle_message(
30             {
31                 "type": "ACK",
32                 "op_id": first_id,
33                 "version": 1,
34                 "op": {"kind": "insert", "pos": 3, "char": "x"},
35             }
36         )
37
38         # ACK 中的权威操作先应用到本地文本，然后 _pump 继续发送队列头。
39         self.assertEqual(state.content, "abcx")
40         self.assertEqual(len(sent), 2)
41         self.assertEqual(sent[1]["op_id"], second_id)
42         self.assertEqual(sent[1]["base_version"], 1)
43
44         state.handle_message(
45             {
46                 "type": "ACK",
47                 "op_id": second_id,
48                 "version": 2,
49                 "op": {"kind": "insert", "pos": 4, "char": "y"},

```

```

49         }
50     )
51
52     self.assertEqual(state.content, "abcxy")
53     # 两个 ACK 都处理完后, 客户端没有 inflight, 也没有待发送操作。
54     self.assertIsNone(state.inflight_op)
55     self.assertEqual(len(state.pending_queue), 0)
56
57     def test_remote_gap_requests_document(self):
58         # 如果收到的远端版本不是当前版本 + 1, 说明中间漏了操作。
59         # 客户端此时不能直接应用, 只能请求完整文档重同步。
60         sent = []
61         state = ClientSyncState("u1", sent.append)
62         state.set_document("", 0)
63         state.handle_message(
64             {
65                 "type": "REMOTE_OP",
66                 "version": 3,
67                 "op": {"kind": "insert", "pos": 0, "char": "z"},
68             }
69         )
70
71         self.assertEqual(sent[-1], {"type": "REQUEST_DOC"})
72
73     def test_welcome_updates_server_assigned_client_id(self):
74         # 服务端可能因为 id 冲突给客户端分配新 id。
75         # 客户端后续生成 op_id 和发送 OP 时必须使用服务端确认后的 id。
76         sent = []
77         state = ClientSyncState("u1", sent.append)
78         state.handle_message({"type": "WELCOME", "client_id": "u2"})
79         state.set_document("", 0)
80         op_id = state.queue_operation(Operation("insert", 0, "x"))
81
82         self.assertEqual(state.client_id, "u2")
83         self.assertEqual(op_id, "u2-1")
84         self.assertEqual(sent[0]["client_id"], "u2")
85
86     def test_chinese_character_is_insertable(self):
87         # 普通中文字符可以插入; Ctrl-S 和 Tab 这类控制字符不能进入文档。
88         self.assertTrue(is_insertable_character("你"))
89         self.assertFalse(is_insertable_character("\x13"))
90         self.assertFalse(is_insertable_character("\t"))
91
92     def test_chinese_display_width_helpers(self):
93         # 终端里中文通常占两个显示列, 光标上下移动要按显示宽度计算。
94         self.assertEqual(text_display_width("a 你 b"), 4)
95         self.assertEqual(display_col_to_char_col("a 你 b", 2), 1)

```



```

96     self.assertEqual(display_col_to_char_col("a 你 b", 3), 2)
97
98     def test_wide_character_backspace_queues_delete(self):
99         # Backspace 不直接修改本地文本，而是排队一个 delete 操作交给服务端确认。
100         sent = []
101         client = TerminalClient("127.0.0.1", 8765, "u1")
102         client.state = ClientSyncState("u1", sent.append)
103         client.state.set_document("ab", 0)
104         client.cursor_pos = 2
105
106         client._handle_key("\x7f")
107
108         self.assertIn("\x7f", BACKSPACE_KEYS)
109         self.assertEqual(sent[0]["op"], {"kind": "delete", "pos": 1})
110         self.assertEqual(client.cursor_pos, 1)
111
112
113 if __name__ == "__main__":
114     unittest.main()

```

服务端输出测试：这一组测试不启动真实 socket，而是检查服务端可观察输出是否稳定。测试覆盖插入、删除、保存、客户端加入和离开事件的格式化结果，同时验证非服务端主进程调用 `shutdown` 时不会错误地关闭父进程资源。

```

1 import unittest
2 from pathlib import Path
3 from unittest.mock import Mock, patch
4
5 from server import CollaborativeServer, format_operation_event, format_save_event,
6     ↪ format_server_event
7
8 class ServerOutputTests(unittest.TestCase):
9     """ 验证服务端终端输出和关闭保护逻辑。
10
11     这些测试不启动真实 socket，而是只检查纯格式化函数和 shutdown 的边界行为。
12     """
13
14     def test_format_insert_operation_event(self):
15         # insert 日志需要额外展示 char，便于观察中文字符等输入是否正确同步。
16         message = format_operation_event(
17             {
18                 "event": "operation",
19                 "server_version": 12,
20                 "client_id": "u1",
21                 "base_version": 11,

```

```

22         "transformed_op": {"kind": "insert", "pos": 5, "char": "你"},
23     }
24 )
25
26 self.assertEqual(message, "OP v12 client=u1 insert pos=5 char='你' base=11")
27
28 def test_format_delete_operation_event(self):
29     # delete 日志没有 char 字段, 只展示操作类型、位置和基准版本。
30     message = format_operation_event(
31         {
32             "event": "operation",
33             "server_version": 13,
34             "client_id": "u2",
35             "base_version": 12,
36             "transformed_op": {"kind": "delete", "pos": 3},
37         }
38     )
39
40     self.assertEqual(message, "OP v13 client=u2 delete pos=3 base=12")
41
42 def test_format_save_event(self):
43     # 客户端触发的保存要在终端输出中标明 client_id。
44     message = format_save_event("requested", "u1", 13, Path("shared.py"))
45
46     self.assertEqual(message, "SAVE requested client=u1 version=13
47     ↪ file=shared.py")
48
49 def test_format_join_leave_events(self):
50     # 加入和退出事件用于课堂演示时观察客户端连接生命周期。
51     self.assertEqual(
52         format_server_event({"event": "join", "client_id": "u1", "address":
53         ↪ "127.0.0.1"}),
54         "JOIN client=u1 address=127.0.0.1",
55     )
56     self.assertEqual(
57         format_server_event({"event": "leave", "client_id": "u1"}),
58         "LEAVE client=u1",
59     )
60
61 def test_shutdown_ignores_non_owner_process(self):
62     # Windows multiprocessing 可能让子进程持有 server 对象副本。
63     # 非 owner 进程调用 shutdown 时必须直接返回, 不能关闭父进程资源。
64     with (
65         patch("server.multiprocessing.Queue", return_value=Mock()),
66         patch("server.multiprocessing.Process", return_value=Mock()),
67     ):
68         server = CollaborativeServer("127.0.0.1", 0, Path("shared.py"))

```

```
67
68     with patch("server.os.getpid", return_value=server._owner_pid + 1):
69         server.shutdown()
70
71     self.assertFalse(server._stop_event.is_set())
72
73
74 if __name__ == "__main__":
75     unittest.main()
```

自动保存备份测试：这一组测试聚焦文件保存行为。通过 mock 隔离真实文件系统后，测试验证手动保存覆盖正式文件前会先复制原文件生成 `.bak` 备份，然后再调用原子写入函数写入新内容。

```
1 import unittest
2 from pathlib import Path
3 from unittest.mock import patch
4
5 from autosave import write_with_backup
6
7
8 class AutosaveTests(unittest.TestCase):
9     """ 验证保存模块中与文件备份相关的可观察行为。 """
10
11     def test_manual_save_writes_backup(self):
12         # 手动保存覆盖正式文件前，应先把原文件复制成 .bak。
13         # 这里用 mock 隔离真实文件系统，只验证调用顺序和目标路径。
14         path = Path("shared.py")
15
16         with (
17             patch.object(Path, "exists", return_value=True),
18             patch("autosave.shutil.copy2") as copy2,
19             patch("autosave.atomic_write_text") as atomic_write,
20         ):
21             write_with_backup(path, "new")
22
23             copy2.assert_called_once_with(path, path.with_name("shared.py.bak"))
24             atomic_write.assert_called_once_with(path, "new")
25
26
27 if __name__ == "__main__":
28     unittest.main()
```

Linux 平台下的基本命令

作业中要求使用 Linux 平台进行操作，我使用一台阿里云 ECS 服务器完成实验。服务器配置为 2 核 2 GB，操作系统为 Ubuntu 24.04。本次实验通过 SSH 连接到服务器，并在真实 Linux 环境中完成命令演示。我使用 Termius 作为 SSH 客户端，便于在 Windows 电脑上远程连接服务器。

本节按照文件和目录、文件内容处理、系统信息、权限管理以及管道组合五类来整理命令。这样比单纯罗列命令更容易看出不同命令的使用场景，也能体现 Linux 命令之间可以组合使用的特点。

1 文件和目录操作

1. 查看当前所在目录：`pwd`
2. 查看当前目录下的文件：`ls`
3. 以详细形式查看文件：`ls -l`
4. 查看隐藏文件：`ls -a`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~# pwd
/root
root@iZbp11a51jwdv4pjbv2r1oZ ~# ls
alist/      clash-for-linux-install/  desktop/  n2n/      snap/
clashctl/   derp/                    frp/      rustDesk/ task2/
root@iZbp11a51jwdv4pjbv2r1oZ ~# ls -l
total 40
drwxr-xr-x  2 root root 4096 Feb  9 14:25 alist/
drwxr-xr-x  7 root root 4096 Mar  2 17:35 clashctl/
drwxr-xr-x  6 root root 4096 Mar  2 17:35 clash-for-linux-install/
drwxr-xr-x  2 root root 4096 Mar  9 19:47 derp/
drwxr-xr-x  2 root root 4096 Apr 27 11:27 desktop/
drwxr-xr-x  3 root root 4096 Feb 21 12:31 frp/
drwxr-xr-x 17 root root 4096 Feb  9 22:49 n2n/
drwxr-xr-x  3 root root 4096 Mar  2 21:18 rustDesk/
drwx----- 3 root root 4096 Mar 14 12:23 snap/
drwxr-xr-x  2 root root 4096 Apr 25 20:50 task2/
root@iZbp11a51jwdv4pjbv2r1oZ ~# ls -a
./      .cache/      desktop/     .pip/        .ssh/
../      clashctl/     frp/         .profile     task2/
alist/   clash-for-linux-install/ .lessht     .pydistutils.cfg .vim/
.bash_history .config/     .local/     rustDesk/    .viminfo
.bashrc    derp/       n2n/        snap/        .wget-hsts
```

图 14 命令 1.1-1.4

5. 创建一个目录：`mkdir linux_test`
6. 进入目录：`cd linux_test`
7. 创建空文件：`touch file1.txt`
8. 向文件写入内容：`echo "hello linux" > file1.txt`

9. 查看文件内容: `cat file1.txt`

10. 复制文件: `cp file1.txt file2.txt`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~# mkdir linux_test
root@iZbp11a51jwdv4pjbv2r1oZ ~# cd linux_test/
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# touch file1.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# echo "hello linux" > file1.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# cat file1.txt
hello linux
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# cp file1.txt file2.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# cat file2.txt
hello linux
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# ls
file1.txt  file2.txt
```

图 15 命令 1.5-1.10

11. 重命名文件: `mv file1.txt newfile.txt`

12. 删除文件: `rm newfile.txt`

13. 返回上一级目录: `cd ..`

14. 删除目录: `rm -r linux_test`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# mv file1.txt newfile.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# ls
file2.txt  newfile.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# rm newfile.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# ls
file2.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# cd ..
root@iZbp11a51jwdv4pjbv2r1oZ ~# ls
alist/      clash-for-linux-install/  desktop/  linux_test/  rustDesk/  task2/
clashctl/   derp/                    frp/      n2n/         snap/
root@iZbp11a51jwdv4pjbv2r1oZ ~# rm -r linux_test/
root@iZbp11a51jwdv4pjbv2r1oZ ~# ls
alist/      clash-for-linux-install/  desktop/  n2n/         snap/
clashctl/   derp/                    frp/      rustDesk/    task2/
```

图 16 命令 1.11-1.14

2 文件内容查看与处理命令

我先使用如下命令创建测试目录和测试文件，然后再对文件内容处理命令进行演示。

```
1 mkdir linux_homework
2 cd linux_homework
3 echo -e "apple\nbanana\napple\norange\nbanana\nlinux\ncommand\npipe" > words.txt
```

1. 查看完整内容: `cat words.txt`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~# mkdir linux_homework
root@iZbp11a51jwdv4pjbv2r1oZ ~# ls
alist/      clash-for-linux-install/  desktop/  linux_homework/  rustDesk/  task2/
clashctl/   derp/                    frp/      n2n/              snap/
root@iZbp11a51jwdv4pjbv2r1oZ ~# cd linux_homework/
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# echo -e "apple\nbanana\napple\norange\nbanana\nlinux\ncommand\npipe" > words.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# ls
words.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# cat words.txt
apple
banana
apple
orange
banana
linux
command
pipe
```

图 17 命令 2.1

2. 查看文件前 3 行: `head -n 3 words.txt`
3. 查看文件后 3 行: `tail -n 3 words.txt`
4. 统计文件行数、单词数、字符数: `wc words.txt`
5. 统计文件行数: `wc -l words.txt`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# head -n 3 words.txt
apple
banana
apple
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# tail -n 3 words.txt
linux
command
pipe
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# wc words.txt
 8  8 52 words.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# wc -l words.txt
8 words.txt
```

图 18 命令 2.2-2.5

6. 对文件内容排序: `sort words.txt`
7. 去除相邻重复行: `uniq words.txt`
8. 查找包含 apple 的行: `grep "apple" words.txt`
9. 查找包含 banana 的行, 并显示行号: `grep -n "banana" words.txt`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# sort words.txt
apple
apple
banana
banana
command
linux
orange
pipe
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# uniq words.txt
apple
banana
apple
orange
banana
linux
command
pipe
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# grep "banana" words.txt
banana
banana
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# grep -n "banana" words.txt
2:banana
5:banana
```

图 19 命令 2.6-2.9

`uniq` 只能去除相邻的重复行，如果重复内容之间隔着其他行，直接执行 `uniq words.txt` 并不能完成全局去重。因此在后面的管道命令中，我又使用 `sort words.txt | uniq`，先把相同内容排到相邻位置，再进行去重统计。

3 系统信息查看命令

1. 查看系统内核和系统信息: `uname -a`
2. 查看当前用户名: `whoami`
3. 查看当前日期时间: `date`
4. 显示日历: `cal`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# uname -a
Linux iZbp11a51jwdv4pjbv2r1oZ 6.8.0-90-generic #91-Ubuntu SMP PREEMPT_DYNAMIC Tue Nov 18 14:14:30 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# whoami
root
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# date
Mon Apr 27 11:36:17 AM CST 2026
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# cal
    April 2026
Su Mo Tu We Th Fr Sa
                1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30
```

图 20 命令 3.1-3.4

如果系统默认没有安装 `cal` 命令，需要先安装 `ncal` 包。我使用 `sudo apt install`

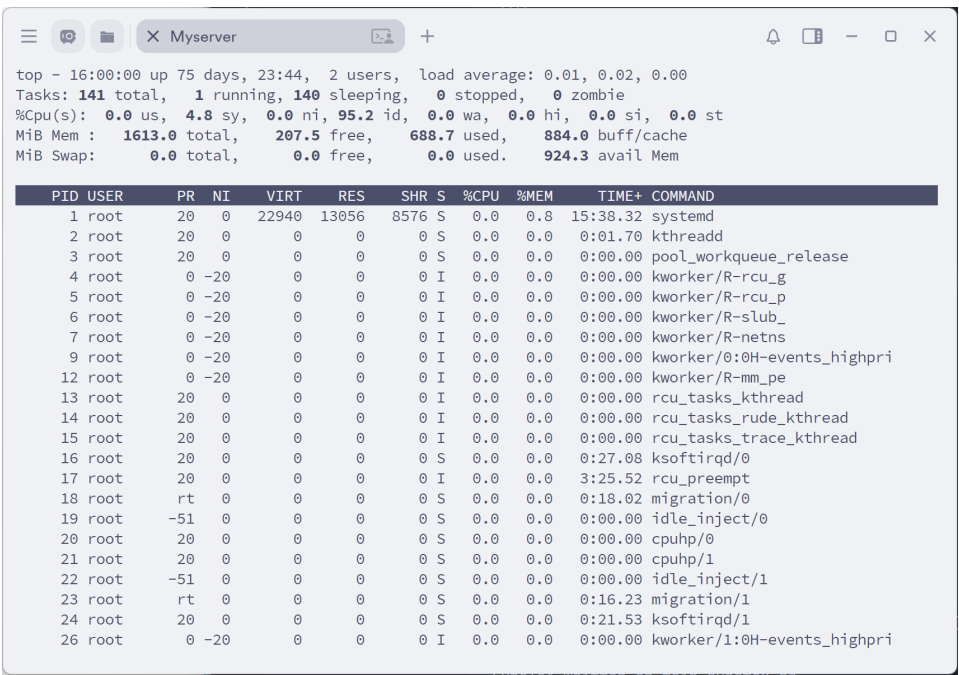
ncal 安装之后，就可以正常显示日历。

- 5. 查看磁盘空间: `df -h`
- 6. 查看内存使用情况: `free -h`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs            162M  1.5M  160M   1% /run
efivarfs         256K  7.4K  244K   3% /sys/firmware/efi/efivars
/dev/vda3        40G   5.8G   32G  16% /
tmpfs            807M    0  807M   0% /dev/shm
tmpfs            5.0M    0   5.0M   0% /run/lock
/dev/vda2       197M   6.2M  191M   4% /boot/efi
overlay          40G   5.8G   32G  16% /var/lib/docker/rootfs/overlayfs/f6fdeb7a0935974bbc5df63
262aeceb1eab665b7deb467c5473618aaa7edeb16
overlay          40G   5.8G   32G  16% /var/lib/docker/rootfs/overlayfs/237c5d820903ac7889c5633
f264a81a5cfd5beb6d68c2e52780dcf004e0fbc2
overlay          40G   5.8G   32G  16% /var/lib/docker/rootfs/overlayfs/8b23e10ea750512823dc7b9
8534178b2485c5248054f51408fb1b2651dcba3a4
overlay          40G   5.8G   32G  16% /var/lib/docker/rootfs/overlayfs/27db7f8296651a194aea4a0
77aldfe1e2ca5ca9a74782f6885fae7792df73f73
tmpfs            162M   12K  162M   1% /run/user/0
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# free -h
              total        used        free      shared  buff/cache   available
Mem:           1.6Gi         651Mi        137Mi         3.3Mi         994Mi        961Mi
Swap:            0B           0B           0B
```

图 21 命令 3.5-3.6

- 7. 查看系统进程和资源占用: `top`



```
top - 16:00:00 up 75 days, 23:44,  2 users,  load average: 0.01, 0.02, 0.00
Tasks: 141 total,  1 running, 140 sleeping,  0 stopped,  0 zombie
%Cpu(s):  0.0 us,  4.8 sy,  0.0 ni, 95.2 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem : 1613.0 total,  207.5 free,  688.7 used,  884.0 buff/cache
MiB Swap:  0.0 total,  0.0 free,  0.0 used,  924.3 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM    TIME+  COMMAND
    1 root        20   0  22940  13056  8576  S   0.0   0.8  15:38.32 systemd
    2 root        20   0     0     0     0  S   0.0   0.0   0:01.70 kthreadd
    3 root        20   0     0     0     0  S   0.0   0.0   0:00.00 pool_workqueue_release
    4 root        0 -20     0     0     0  I   0.0   0.0   0:00.00 kworker/R-rcu_g
    5 root        0 -20     0     0     0  I   0.0   0.0   0:00.00 kworker/R-rcu_p
    6 root        0 -20     0     0     0  I   0.0   0.0   0:00.00 kworker/R-slub_
    7 root        0 -20     0     0     0  I   0.0   0.0   0:00.00 kworker/R-netns
    9 root        0 -20     0     0     0  I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
   12 root        0 -20     0     0     0  I   0.0   0.0   0:00.00 kworker/R-mm_pe
   13 root        20   0     0     0     0  I   0.0   0.0   0:00.00 rcu_tasks_kthread
   14 root        20   0     0     0     0  I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
   15 root        20   0     0     0     0  I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
   16 root        20   0     0     0     0  S   0.0   0.0   0:27.08 ksoftirqd/0
   17 root        20   0     0     0     0  I   0.0   0.0   3:25.52 rcu_preempt
   18 root        rt    0     0     0     0  S   0.0   0.0   0:18.02 migration/0
   19 root       -51   0     0     0     0  S   0.0   0.0   0:00.00 idle_inject/0
   20 root        20   0     0     0     0  S   0.0   0.0   0:00.00 cpuhp/0
   21 root        20   0     0     0     0  S   0.0   0.0   0:00.00 cpuhp/1
   22 root       -51   0     0     0     0  S   0.0   0.0   0:00.00 idle_inject/1
   23 root        rt    0     0     0     0  S   0.0   0.0   0:16.23 migration/1
   24 root        20   0     0     0     0  S   0.0   0.0   0:21.53 ksoftirqd/1
   26 root        0 -20     0     0     0  I   0.0   0.0   0:00.00 kworker/1:0H-events_highpri
```

图 22 命令 3.7

- 8. 查看当前系统进程: `ps aux`


```

root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.7  22940 13056 ?        Ss   Feb08   15:38 /usr/lib/systemd/systemd --system --
root         2  0.0  0.0      0     0 ?        S    Feb08    0:01 [kthreadd]
root         3  0.0  0.0      0     0 ?        S    Feb08    0:00 [pool_workqueue_release]
root         4  0.0  0.0      0     0 ?        I<   Feb08    0:00 [kworker/R-rcu_g]
root         5  0.0  0.0      0     0 ?        I<   Feb08    0:00 [kworker/R-rcu_p]
root         6  0.0  0.0      0     0 ?        I<   Feb08    0:00 [kworker/R-slub_]
root         7  0.0  0.0      0     0 ?        I<   Feb08    0:00 [kworker/R-netns]
root         9  0.0  0.0      0     0 ?        I<   Feb08    0:00 [kworker/0:0H-events_highpri]
root        12  0.0  0.0      0     0 ?        I<   Feb08    0:00 [kworker/R-mm_pe]
root        13  0.0  0.0      0     0 ?        I    Feb08    0:00 [rcu_tasks_kthread]
root        14  0.0  0.0      0     0 ?        I    Feb08    0:00 [rcu_tasks_rude_kthread]
root        15  0.0  0.0      0     0 ?        I    Feb08    0:00 [rcu_tasks_trace_kthread]
root        16  0.0  0.0      0     0 ?        S    Feb08    0:27 [ksoftirqd/0]
root        17  0.0  0.0      0     0 ?        I    Feb08    3:25 [rcu_preempt]
root        18  0.0  0.0      0     0 ?        S    Feb08    0:18 [migration/0]
root        19  0.0  0.0      0     0 ?        S    Feb08    0:00 [idle_inject/0]
root        20  0.0  0.0      0     0 ?        S    Feb08    0:00 [cpuhp/0]
root        21  0.0  0.0      0     0 ?        S    Feb08    0:00 [cpuhp/1]
root        22  0.0  0.0      0     0 ?        S    Feb08    0:00 [idle_inject/1]
root        23  0.0  0.0      0     0 ?        S    Feb08    0:16 [migration/1]
root        24  0.0  0.0      0     0 ?        S    Feb08    0:21 [ksoftirqd/1]
root        26  0.0  0.0      0     0 ?        I<   Feb08    0:00 [kworker/1:0H-events_highpri]
root        27  0.0  0.0      0     0 ?        S    Feb08    0:00 [kdevtmpfs]
root        28  0.0  0.0      0     0 ?        I<   Feb08    0:00 [kworker/R-inet_]
root        29  0.0  0.0      0     0 ?        S    Feb08    0:02 [kauditd]
root        31  0.0  0.0      0     0 ?        S    Feb08    0:04 [khungtaskd]
root        33  0.0  0.0      0     0 ?        S    Feb08    0:00 [oom_reaper]

```

图 23 命令 3.8

由于使用的是远程服务器，系统进程比较多，截图中只保留了与本次演示相关的部分。

9. 查看历史命令: `history`

```

801  uname -a
802  whoami
803  date
804  cal
805  apt install ncal
806  cal
807  clear
808  uname -a
809  whoami
810  date
811  cal
812  clea
813  clear
814  df -h
815  top
816  df -h
817  clear
818  df -h
819  clear
820  top
821  clear
822  free -h
823  df -h
824  free -h
825  clear
826  ps aux
827  clear
828  history

```

图 24 命令 3.9

历史记录中包含了我之前使用过的命令，包括一些测试命令和安装命令等。这里只截了本次课程展示的几个历史命令。

4 权限相关命令

1. 创建脚本文件: `touch test.sh`
2. 查看文件权限: `ls -l test.sh`
3. 给文件添加可执行权限: `chmod +x test.sh`
4. 再次查看权限变化: `ls -l test.sh`
5. 查看文件详细状态信息: `stat test.sh`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# touch test.sh
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# ls -l test.sh
-rw-r--r-- 1 root root 0 Apr 27 11:38 test.sh
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# chmod +x test.sh
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# ls -l test.sh
-rwxr-xr-x 1 root root 0 Apr 27 11:38 test.sh*
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# stat test.sh
  File: test.sh
  Size: 0          Blocks: 0          IO Block: 4096   regular empty file
Device: 253,3    Inode: 1179141    Links: 1
Access: (0755/-rwxr-xr-x)  Uid: ( 0/   root)   Gid: ( 0/   root)
Access: 2026-04-27 11:38:00.613626249 +0800
Modify: 2026-04-27 11:38:00.613626249 +0800
Change: 2026-04-27 11:38:14.493042638 +0800
 Birth: 2026-04-27 11:38:00.613626249 +0800
```

图 25 命令 4.1-4.5

5 管道符组合命令

管道符 `|` 的作用是将前一个命令的输出结果作为后一个命令的输入。通过管道符，可以把多个简单命令组合成一个更复杂的数据处理流程。例如 `cat words.txt | sort | uniq -c | sort -nr` 先读取文件内容，再进行排序，然后统计重复行出现次数，最后按照出现次数从高到低排序，体现了 Linux 命令组合使用的灵活性。

1. 把文件内容传给 `sort` 排序后去重: `cat words.txt | sort | uniq`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# cat words.txt | sort | uniq
apple
banana
command
linux
orange
pipe
```

图 26 命令 5.1

2. 统计每个单词出现次数: `cat words.txt | sort | uniq -c`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# cat words.txt | sort | uniq -c
  2 apple
  2 banana
  1 command
  1 linux
  1 orange
  1 pipe
```

图 27 命令 5.2

3. 找出包含字母 a 的行，并排序: `grep "a" words.txt | sort`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# cat words.txt | grep "a" | sort
apple
apple
banana
banana
command
orange
```

图 28 命令 5.3

4. 在当前目录中筛选.txt 文件: `ls | grep '\.txt$'`
5. 在进程列表中查找 bash 相关进程: `ps aux | grep bash`
6. 查看最近使用的命令: `history | tail`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# ls -l | grep ".txt"
-rw-r--r-- 1 root root 52 Apr 27 11:33 words.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# ps aux | grep bash
root    219565  0.0  0.0  2616  128 ?        Ss   Mar09   0:00 /bin/sh -c bash /app/build_
cert.sh $DERP_HOST $DERP_CERTS /app/san.conf && /app/derper --hostname=$DERP_HOST --ce
rtmode=manual --certdir=$DERP_CERTS --stun=$DERP_STUN --a=$DERP_ADDR --http-p
ort=$DERP_HTTP_PORT --verify-clients=$DERP_VERIFY_CLIENTS
root    243312  0.0  0.2  9052  4480 tty1      S+   Mar14   0:00 -bash
root    578530  0.0  0.1  6544  2304 pts/0    S+   11:40   0:00 grep --color=auto bash
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# history | tail
/etc/ssh/sshd_config | grep -E '^Port'
sudo systemctl status ssh
service ssh restart
ufw status
systemctl restart ssh
systemctl status ssh
sudo systemctl list-units --type=service | grep ssh
systemctl restart sshd
cd root
ranger
```

图 29 命令 5.4-5.6

7. 筛选磁盘挂载信息: `df -h | grep "/dev"`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~ /linux_homework# df -h | grep "/"
tmpfs          162M   1.5M   160M    1% /run
efivarfs       256K   7.4K   244K    3% /sys/firmware/efi/efivars
/dev/vda3      40G    5.8G   32G    16% /
tmpfs          807M    0   807M    0% /dev/shm
tmpfs          5.0M    0   5.0M    0% /run/lock
/dev/vda2     197M    6.2M   191M    4% /boot/efi
overlay        40G    5.8G   32G    16% /var/lib/docker/rootfs/overlayfs/f6fdeb7a0935974bbc5df63
262aeeeb1eab665b7deb467c5473618aaa7edeb16
overlay        40G    5.8G   32G    16% /var/lib/docker/rootfs/overlayfs/237c5d820903ac7889c5633
f264a81a5cfd5beb6d68c2e52780dcf04e0fbc2
overlay        40G    5.8G   32G    16% /var/lib/docker/rootfs/overlayfs/8b23e10ea750512823dc7b9
8534178b2485c5248054f51408fb1b2651dcba3a4
overlay        40G    5.8G   32G    16% /var/lib/docker/rootfs/overlayfs/27db7f8296651a194aea4a0
77a1dfe1e2ca5ca9a74782f6885fae7792df73f73
tmpfs          162M   12K   162M    1% /run/user/0
```

图 30 命令 5.7

8. 统计文件行数: `cat words.txt | wc -l`
9. 统计单词出现次数,并按出现次数从高到低排序:`cat words.txt | sort | uniq`
`-c | sort -nr`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~ /linux_homework# cat words.txt | wc -l
8
root@iZbp11a51jwdv4pjbv2r1oZ ~ /linux_homework# cat words.txt | sort | uniq -c | sort -nr
      2 banana
      2 apple
      1 pipe
      1 orange
      1 linux
      1 command
```

图 31 命令 5.8-5.9

6 实验小结

通过本次基础命令实验,我对 Linux 命令行的工作方式有了更系统的理解。文件和目录命令主要解决“在哪里”和“有什么”的问题,内容查看与处理命令用于观察和整理文本,系统信息命令帮助了解当前运行环境,权限命令则体现了 Linux 中文件可读、可写、可执行权限的基本模型。

其中管道命令给我的印象最深。单个命令的功能通常比较简单,但通过 `|` 可以把多个命令连接成完整的数据处理流程。例如排序、去重、计数和再次排序可以组合成一条命令完成词频统计。这体现了 Linux 工具“每个程序只做好一件事,再通过组合完成复杂任务”的思想,也为后续 Shell 脚本编程打下了基础。

Linux 下的 Shell 编程案例

本次作业我设计并实现了一个基于 Bash Shell 的 Linux 命令答题系统。程序通过菜单提供开始答题、查看历史成绩、清空历史成绩和退出系统等功能，题目从外部题库文件中读取，答题结束后会把本次成绩保存到成绩文件中。这个案例既能练习 Linux 基础命令，也能体现 Shell 脚本中的条件判断、循环控制、函数封装、文件读写和程序调试等知识点。相比只演示单条命令，答题系统需要把输入、判断、统计、持久化和异常处理组织成完整流程，更能体现 Shell 脚本在 Linux 自动化任务中的作用。

1 程序功能说明

本程序的主要功能如下：

- 显示主菜单，用户可以通过输入编号选择不同功能。
- 从 `questions.txt` 题库文件中逐行读取题目和标准答案。
- 判断用户输入是否为空、是否与标准答案一致，并根据结果计算得分和错误次数。
- 当答错次数达到设定上限时，系统自动结束本轮答题。
- 将每次答题的时间、得分、已答题数和错误题数保存到 `score.txt` 文件。
- 支持查看历史成绩，并使用 `awk` 统计历史最高分。
- 支持清空历史成绩，清空前需要用户再次确认，避免误操作。

程序采用“主菜单 + 功能函数”的结构，整体逻辑比较清晰。用户进入程序后，主循环会不断显示菜单，直到用户选择退出。每个功能被封装成独立函数，例如 `init_env`、`show_menu`、`start_quiz`、`save_score`、`show_history` 和 `clear_history`，这样可以减少重复代码。

2 Shell 脚本执行方法

在 Linux 平台下，可以进入脚本所在目录后执行以下命令运行程序：

```
1 cd task2
2 chmod +x linux_quiz.sh
3 ./linux_quiz.sh
```

其中，`chmod +x linux_quiz.sh` 用于给脚本添加可执行权限，`./linux_quiz.sh` 表示执行当前目录下的脚本文件。

3 程序实现思路

脚本整体采用函数化结构，把初始化环境、显示菜单、开始答题、保存成绩、查看历史成绩和清空历史成绩拆分为多个函数。主程序只负责循环显示菜单并根据用户选择调用对应函数，这样可以避免所有逻辑堆在一个长脚本中，也方便后续单独修改某个功能。

题库文件使用 `|` 作为题目和答案之间的分隔符，每一行表示一道题。答题函数通过 `while IFS='|' read -r question correct_answer` 逐行读取题库，并使用独立文件描述符读取文件，避免用户输入和题库读取互相干扰。用户输入后，脚本会先去掉首尾空格，再判断是否为空、是否与标准答案一致，并更新分数、已答题数和错误次数。当错误次数达到上限时，当前轮答题自动结束。

成绩文件使用普通文本保存，每条记录包含答题时间、得分、已答题数和错误题数。查看历史成绩时，脚本使用 `awk` 从成绩文件中统计最高分；清空历史成绩前会要求用户输入确认，避免误删。这样既保持了脚本实现的简单性，也展示了 Shell 脚本对文本文件和命令行工具的组合能力。

4 案例展示

与上一次基础命令作业一致，本次 Shell 脚本也在 Ubuntu 服务器上运行和测试，后续调试同样在这台服务器上完成。

首先创建一个目录来存放本次作业，然后放入 Shell 脚本文件和 `txt` 题库文件。

```
root@iZbp11a51jwdv4pjbv2r1oZ ~# ls
alist/  clashctl/  clash-for-linux-install/  derp/  frp/  linux_homework/  n2n/  rustDesk/  snap/
root@iZbp11a51jwdv4pjbv2r1oZ ~# mkdir task2
root@iZbp11a51jwdv4pjbv2r1oZ ~# ls
alist/  clash-for-linux-install/  frp/  linux_homework/  n2n/  snap/
clashctl/  derp/  linux_homework/  rustDesk/  task2/
root@iZbp11a51jwdv4pjbv2r1oZ ~# cd task2/
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# touch linux_quiz.sh
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# vim linux_quiz.sh
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# touch questions.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# vim questions.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# ls
linux_quiz.sh  questions.txt
```

随后按照前面的执行方法运行脚本。

主菜单页面：



开始答题界面:



图 32 答题界面

答题结束界面:

```
-----  
第 11 题：查看当前登录用户  
请输入答案：uname  
回答错误！正确答案是：whoami  
当前得分：9，错误次数：2/2  
  
错误次数达到 2 次，答题结束。  
  
===== 答题结果 =====  
本次得分：9  
已答题数：11  
错误题数：2  
成绩已保存到 ./score.txt  
  
按 Enter 键返回主菜单...
```

图 33 答题结束界面

历史成绩界面：

```
===== 历史成绩 =====  
最高分：24  
-----  
时间 | 得分 | 已答题数 | 错误题数  
-----  
2026-04-25 21:07:00 | 1 | 1 | 2  
2026-04-27 13:17:50 | 9 | 11 | 2  
2026-04-27 13:20:38 | 1 | 1 | 2  
2026-04-27 13:29:59 | 24 | 25 | 1  
  
按 Enter 键返回主菜单...
```

图 34 历史成绩

题库和历史成绩都保存在当前目录下的 `txt` 文件中。目前题库包含 25 道题，后续如果需要扩展题库，可以直接在 `questions.txt` 文件中添加题目和答案，格式为“题目 | 答案”，每行一题。


```
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# ls
linux_quiz.sh*  questions.txt*  score.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# cat score.txt
2026-04-25 21:07:00|1|1|2
2026-04-27 13:17:50|9|11|2
2026-04-27 13:20:38|1|1|2
2026-04-27 13:29:59|24|25|1
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# cat questions.txt
查看当前所在目录|pwd
查看当前目录下文件和文件夹|ls
切换目录|cd
创建目录|mkdir
创建空文件|touch
查看文件内容|cat
```

图 35 题库和历史成绩文本文件

清空历史成绩时，程序会要求用户确认，可以选择确认清空或取消操作。

<pre>===== Linux 命令答题系统 ===== 1. 开始答题 2. 查看历史成绩 3. 清空历史成绩 0. 退出系统 ===== PS: 答错 2 道题后系统会自动结束 请输入功能编号: 3 确认清空历史成绩吗? 输入 y 确认: n 已取消清空操作。 按 Enter 键返回主菜单...■</pre>	<pre>===== Linux 命令答题系统 ===== 1. 开始答题 2. 查看历史成绩 3. 清空历史成绩 0. 退出系统 ===== PS: 答错 2 道题后系统会自动结束 请输入功能编号: 3 确认清空历史成绩吗? 输入 y 确认: y 历史成绩已清空。 按 Enter 键返回主菜单...■</pre>
--	--

图 36 清空历史成绩

退出系统。

```
=====
          Linux 命令答题系统
=====
1. 开始答题
2. 查看历史成绩
3. 清空历史成绩
0. 退出系统
=====

PS: 答错 2 道题后系统会自动结束

请输入功能编号: 0
系统已退出。
```

图 37 退出系统

5 Shell 程序调试方法

在开发和调试 Shell 脚本时，我主要使用 `bash -n linux_quiz.sh` 命令来检查脚本的语法错误。这个命令会解析脚本但不执行它，可以快速发现语法问题。

正常情况下，命令行没有输出，表示没有发现语法错误。

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# bash -n linux_quiz.sh
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2#
```

如果有错误，会显示错误信息和所在行号，方便定位问题。

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# vim linux_quiz.sh
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# bash -n linux_quiz.sh
linux_quiz.sh: line 143: syntax error near unexpected token `}'
linux_quiz.sh: line 143: `}'
```

使用 `vim` 打开脚本后可以看到，错误原因是分支结构中少了一个 `fi`，导致语法检查失败。补全该关键字后再次运行检查，脚本就不再报错。

```
# 查看历史成绩，并在最上方显示最高分
show_history() {
    echo
    echo "===== 历史成绩 ====="

    if [ ! -f "$SCORE_FILE" ]; then
        echo "暂无成绩文件。"
        return
    fi

    if [ ! -s "$SCORE_FILE" ]; then
        echo "暂无历史成绩。"
        return
    fi
}
```

图 38 错误修正

完成 Shell 脚本之后，可以使用 `bash -x linux_quiz.sh` 来执行脚本并显示每条命令的执行过程，这样可以更细致地调试程序逻辑和变量值。



```
+ show_menu
+ echo

+ echo =====
+ echo '          Linux 命令答题系统'
+ echo '          Linux 命令答题系统'
+ echo =====
+ echo '1. 开始答题'
1. 开始答题
+ echo '2. 查看历史成绩'
2. 查看历史成绩
+ echo '3. 清空历史成绩'
3. 清空历史成绩
+ echo '0. 退出系统'
0. 退出系统
+ echo =====
+ echo

+ echo 'PS: 答错 2 道题后系统会自动结束'
PS: 答错 2 道题后系统会自动结束
+ echo

+ read -r -p 请输入功能编号: choice
请输入功能编号: 
```

图 39 执行跟踪调试

另外，我还构造了一些异常输入场景进行测试，例如删除 `questions.txt`、输入非法选项等，用来验证程序的健壮性和错误处理能力。

删除 `questions.txt` 后运行程序：

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# ls
linux_quiz.sh*  questions.txt*  score.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# rm questions.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# ls
linux_quiz.sh*  score.txt
```

图 40 删除题库文件

```
题库文件不存在: ./questions.txt

按 Enter 键返回主菜单...
```

图 41 运行程序

可以看出程序并没有崩溃，而是提示了题库文件不存在，说明程序的错误处理机制是有效的。

恢复题库文件后，答题输入中文选项：

```
-----
第 1 题：查看当前所在目录
请输入答案：中文
回答错误！正确答案是：pwd
当前得分：0，错误次数：1/2

按 Enter 继续下一题...
```

图 42 输入中文选项

程序能够正确识别并说明该答案错误，说明程序对输入的处理也是正确的。

在主菜单页面输入非法选项时，程序不会崩溃，而是继续循环并提示用户输入正确选项。

```
=====
Linux 命令答题系统
=====
1. 开始答题
2. 查看历史成绩
3. 清空历史成绩
0. 退出系统
=====

PS: 答错 2 道题后系统会自动结束

请输入功能编号: asdads
```

图 43 输入非法选项

6 实验小结

这个 Shell 编程案例把前一节学到的基础命令进一步组织成了一个可交互的小程序。脚本中既有 `read` 获取用户输入，也有 `if` 和 `case` 完成分支判断，还有 `while` 循环读取题库文件，并通过重定向把成绩追加到文本文件中。相比单独练习命令，完整脚本更强调流程设计和异常处理。

通过调试过程可以看到，Shell 脚本虽然语法相对简洁，但对符号和结构非常敏感，例如缺少 `fi` 就会导致语法检查失败。因此在编写脚本时，先用 `bash -n` 检查语法，再用 `bash -x` 跟踪执行过程，是比较实用的调试方法。后续如果继续完善这个答题系统，还可以增加随机出题、错题记录、分数排行榜和题库合法性检查等功能。

7 源代码与题库展示

以下是本次 Shell 编程作业的主要源代码和配套题库文件。其中，`linux_quiz.sh` 负责菜单交互、答题流程、成绩保存和异常处理，`questions.txt` 则保存题目与标准答案。题库文件采用“题目 | 答案”的格式组织，每行对应一道题。题库文件可以根据需要进行扩展，添加更多题目和答案。

7.1 Shell 脚本

```
1 #!/usr/bin/env bash
2
```

```
3 QUESTION_FILE="./questions.txt"
4 SCORE_FILE="./score.txt"
5 WRONG_LIMIT=2
6
7 # 初始化运行环境
8 init_env() {
9     if [ ! -f "$SCORE_FILE" ]; then
10         touch "$SCORE_FILE"
11     fi
12 }
13
14 # 暂停，便于用户查看结果
15 pause_screen() {
16     echo
17     read -r -p " 按 Enter 键返回主菜单..."
18 }
19
20 # 显示主菜单
21 show_menu() {
22     echo
23     echo "===== "
24     echo "      Linux 命令答题系统"
25     echo "===== "
26     echo "1. 开始答题"
27     echo "2. 查看历史成绩"
28     echo "3. 清空历史成绩"
29     echo "0. 退出系统"
30     echo "===== "
31     echo
32     echo "PS: 答错 $WRONG_LIMIT 道题后系统会自动结束"
33     echo
34 }
35
36 # 核心答题功能
37 start_quiz() {
38     score=0
39     wrong=0
40     total=0
41     answered=0
42
43     if [ ! -f "$QUESTION_FILE" ]; then
44         echo " 题库文件不存在: $QUESTION_FILE"
45         return
46     fi
47
48     # 使用文件描述符 3 读取题库，避免用户输入被题库文件提前消费。
49     while IFS='|' read -r question correct_answer <&3; do
```

```
50     # 跳过空行
51     if [ -z "$question" ]; then
52         continue
53     fi
54
55     total=$((total + 1))
56
57     echo "-----"
58     echo " 第 $total 题: $question"
59     read -r -p " 请输入答案: " user_answer
60
61     # 去掉用户输入前后的空格
62     user_answer="$(echo "$user_answer" | xargs)"
63
64     if [ -z "$user_answer" ]; then
65         echo " 答案不能为空, 本题判定为错误。"
66         wrong=$((wrong + 1))
67     elif [ "$user_answer" = "$correct_answer" ]; then
68         echo " 回答正确, +1 分。"
69         score=$((score + 1))
70         answered=$((answered + 1))
71     else
72         echo " 回答错误! 正确答案是: $correct_answer"
73         wrong=$((wrong + 1))
74         answered=$((answered + 1))
75     fi
76
77     echo " 当前得分: $score, 错误次数: $wrong/$WRONG_LIMIT"
78     echo
79
80     if [ "$wrong" -ge "$WRONG_LIMIT" ]; then
81         echo
82         echo " 错误次数达到 $WRONG_LIMIT 次, 答题结束。"
83         break
84     fi
85
86     read -r -p " 按 Enter 继续下一题..."
87     clear
88
89     done 3< "$QUESTION_FILE"
90
91     echo
92     echo "===== 答题结果 ====="
93     echo " 本次得分: $score"
94     echo " 已答题数: $answered"
95     echo " 错误题数: $wrong"
96
```

```

97     save_score "$score" "$answered" "$wrong"
98 }
99
100 # 保存成绩
101 save_score() {
102     current_score="$1"
103     answered_count="$2"
104     wrong_count="$3"
105     current_time="$(date '+%Y-%m-%d %H:%M:%S')"
106
107     echo "$current_time|$current_score|$answered_count|$wrong_count" >> "$SCORE_FILE"
108
109     if [ -w "$SCORE_FILE" ]; then
110         echo " 成绩已保存到 $SCORE_FILE"
111     else
112         echo " 成绩文件不可写，保存失败。"
113     fi
114 }
115
116 # 查看历史成绩，并在最上方显示最高分
117 show_history() {
118     echo
119     echo "===== 历史成绩 ====="
120
121     if [ ! -f "$SCORE_FILE" ]; then
122         echo " 暂无成绩文件。"
123         return
124     fi
125
126     if [ ! -s "$SCORE_FILE" ]; then
127         echo " 暂无历史成绩。"
128         return
129     fi
130
131     highest_score="$(awk -F'|' 'BEGIN {max=0} {if ($2 > max) max=$2} END {print
↵ max}' "$SCORE_FILE")"
132
133     echo " 最高分: $highest_score"
134     echo "-----"
135     echo " 时间 | 得分 | 已答题数 | 错误题数"
136     echo "-----"
137
138     while IFS='|' read -r time score answered wrong; do
139         if [ -n "$time" ]; then
140             echo "$time | $score | $answered | $wrong"
141         fi
142     done < "$SCORE_FILE"

```



```
143 }
144
145 # 清空历史成绩
146 clear_history() {
147     echo
148     read -r -p " 确认清空历史成绩吗? 输入 y 确认: " confirm
149
150     if [ "$confirm" = "y" ] || [ "$confirm" = "Y" ]; then
151         > "$SCORE_FILE"
152         echo " 历史成绩已清空。"
153     else
154         echo " 已取消清空操作。"
155     fi
156 }
157
158 # 主程序入口
159 init_env
160
161 while true; do
162     clear
163     show_menu
164     read -r -p " 请输入功能编号: " choice
165
166     case "$choice" in
167         1)
168             clear
169             start_quiz
170             pause_screen
171             ;;
172         2)
173             clear
174             show_history
175             pause_screen
176             ;;
177         3)
178             clear_history
179             pause_screen
180             ;;
181         0)
182             echo " 系统已退出。"
183             exit 0
184             ;;
185         *)
186             echo " 输入错误, 请输入 0 到 3 之间的数字。"
187             ;;
188     esac
189 done
```

7.2 题库文件

```
1 查看当前所在目录 |pwd
2 查看当前目录下文件和文件夹 |ls
3 切换目录 |cd
4 创建目录 |mkdir
5 创建空文件 |touch
6 查看文件内容 |cat
7 复制文件 |cp
8 移动文件或重命名文件 |mv
9 删除文件 |rm
10 显示当前日期和时间 |date
11 查看当前登录用户 |whoami
12 查看系统内核信息 |uname
13 查找文件内容中指定关键词 |grep
14 统计文件行数、单词数和字符数 |wc
15 对文本内容进行排序 |sort
16 去除相邻重复行 |uniq
17 查看文件前几行内容 |head
18 查看文件后几行内容 |tail
19 查看磁盘空间使用情况 |df
20 查看内存使用情况 |free
21 查看当前进程信息 |ps
22 修改文件权限 |chmod
23 查看历史命令 |history
24 清屏 |clear
25 查看命令帮助文档 |man
```